MySQL8

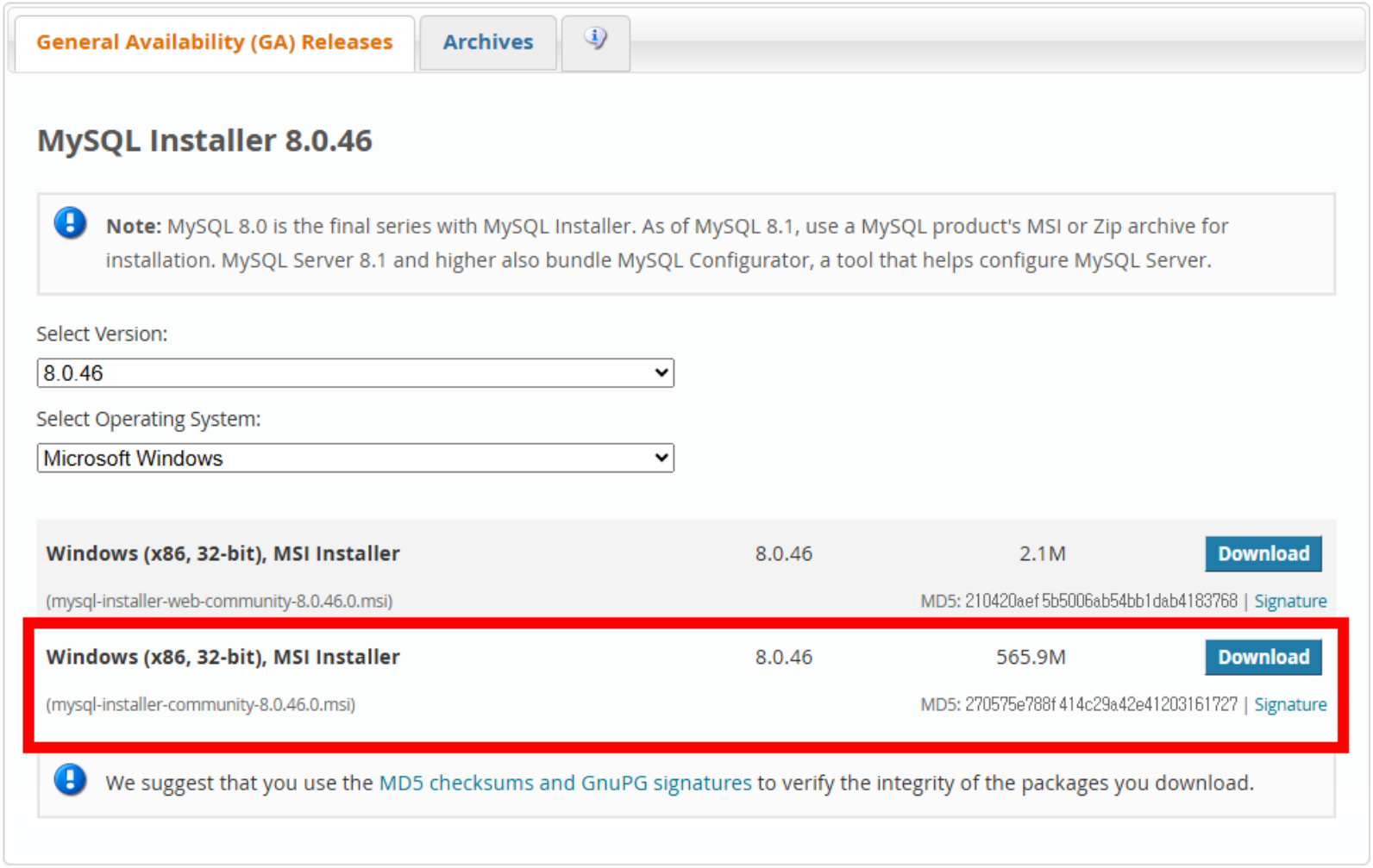
inky4832@daum.net

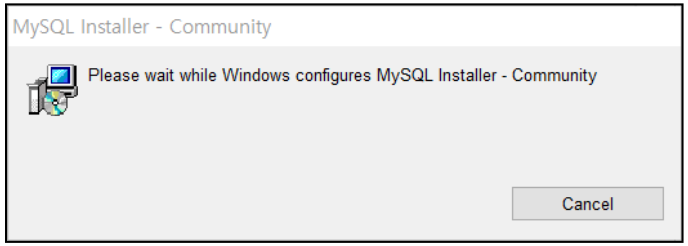
chapter 01.

MySQL8 설치

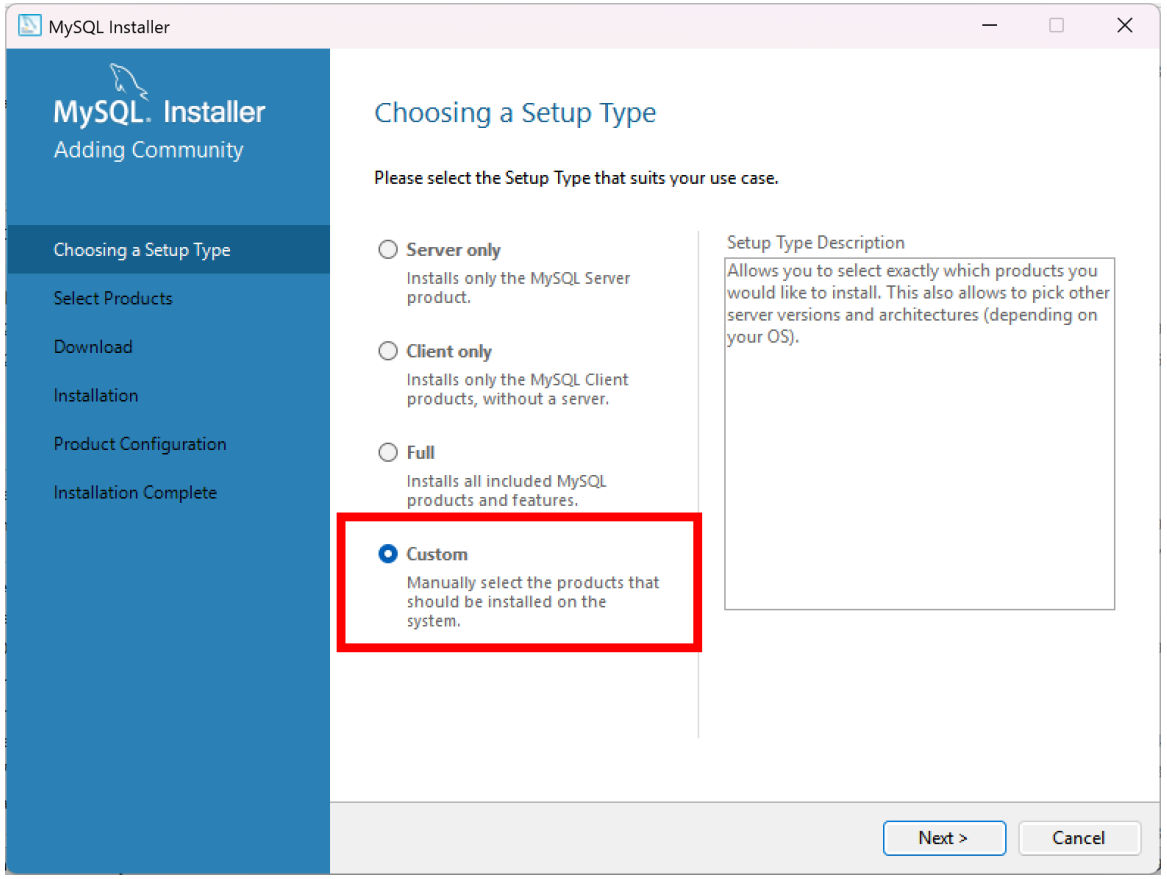
❑ MySQL8 설치

https://dev.mysql.com/downloads/installer/

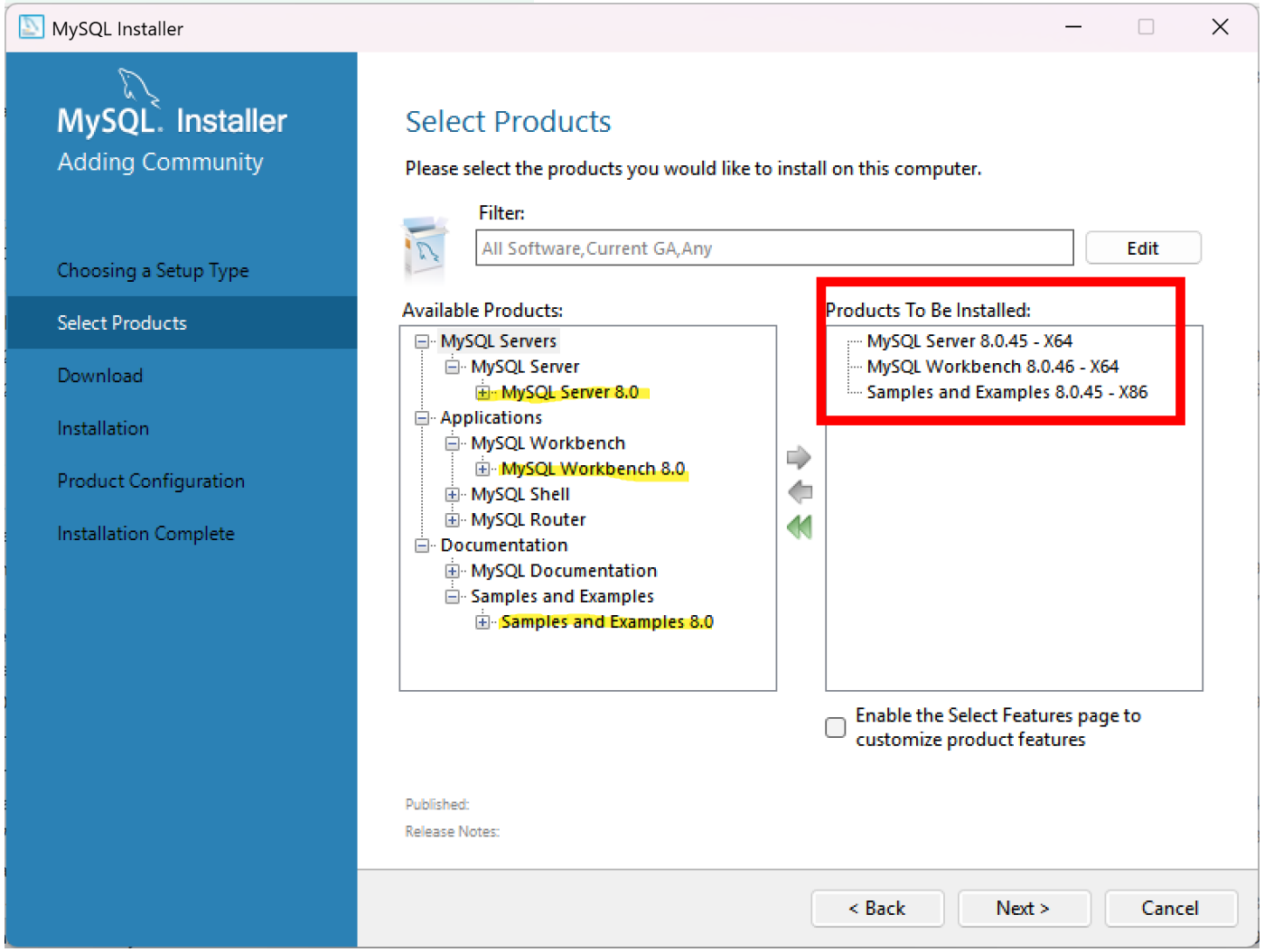



MySQL Installer - Community
Please wait while Windows configures MySQL Installer - Community
Cancel

Custom 선택

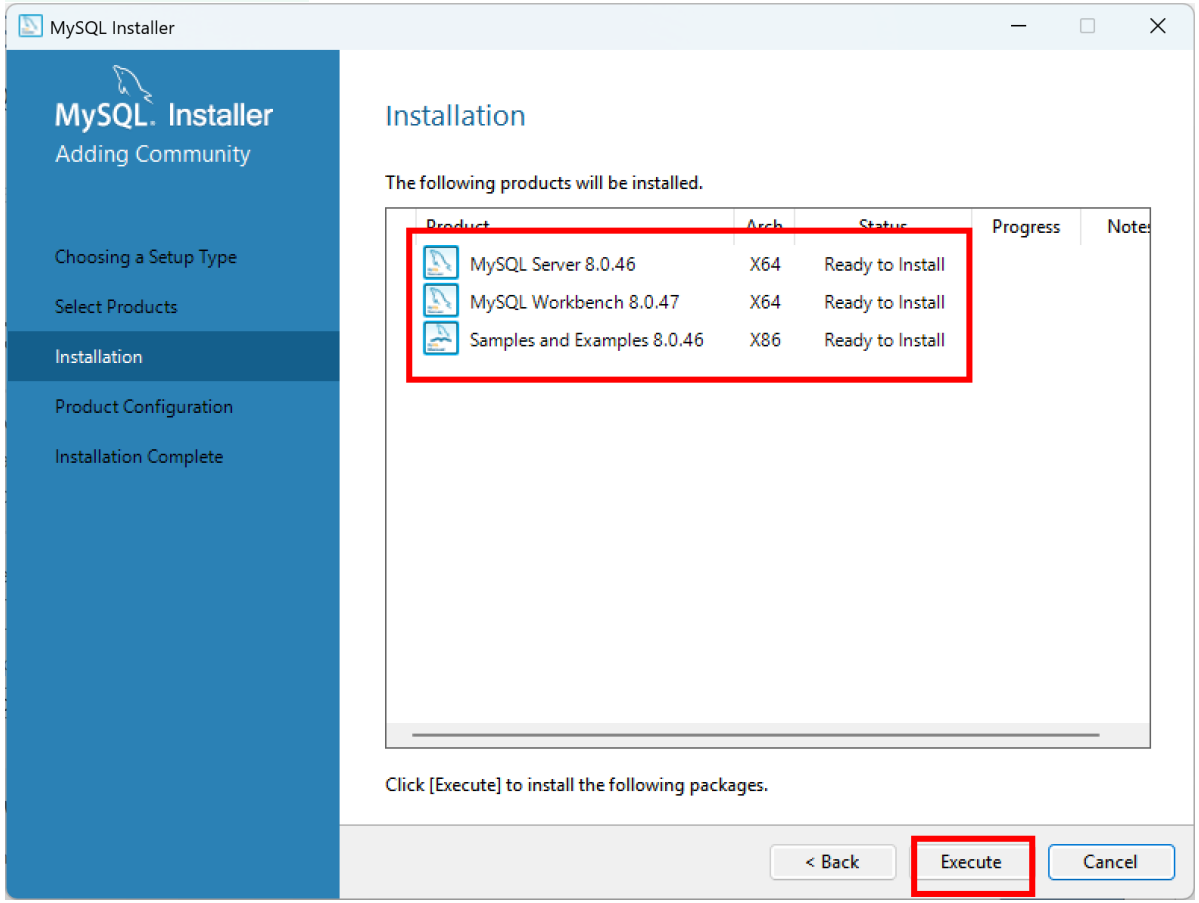

MySQL Installer
MySQL. Installer
Adding Community
Choosing a Setup Type
Select Products
Download
Installation
Product Configuration
Installation Complete
Choosing a Setup Type
Please select the Setup Type that suits your use case.
Server only
Installs only the MySQL Server product.
Client only
Installs only the MySQL Client products, without a server.
Full
Installs all included MySQL products and features.
Custom
Manually select the products that should be installed on the system.
Setup Type Description
Allows you to select exactly which products you would like to install. This also allows to pick other server versions and architectures (depending on your OS).
Next >
Cancel

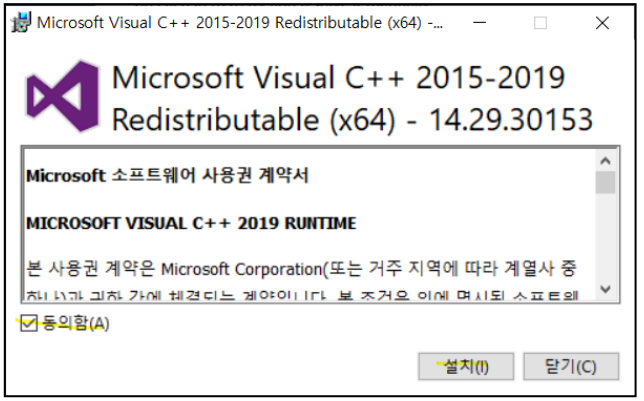
Server/Workbench/Sample 선택



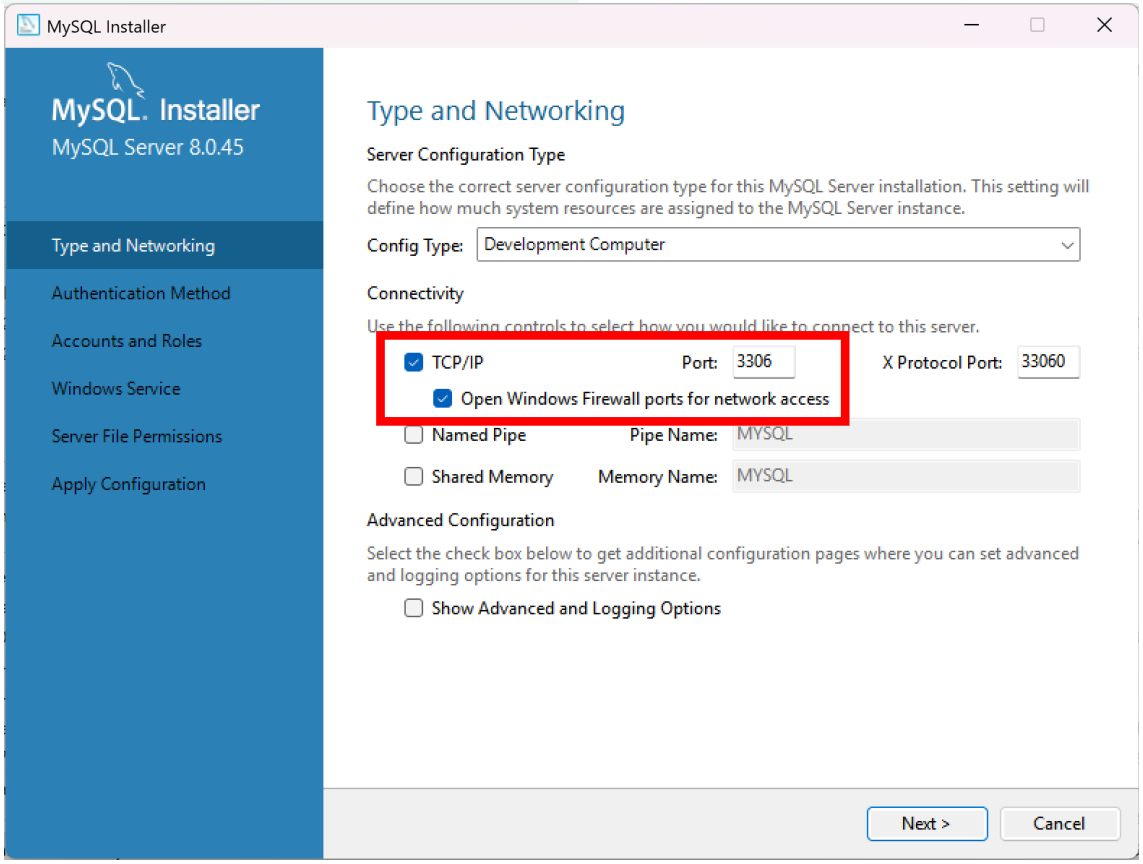
❑ MySQL8 설치

필수 S/W 설치

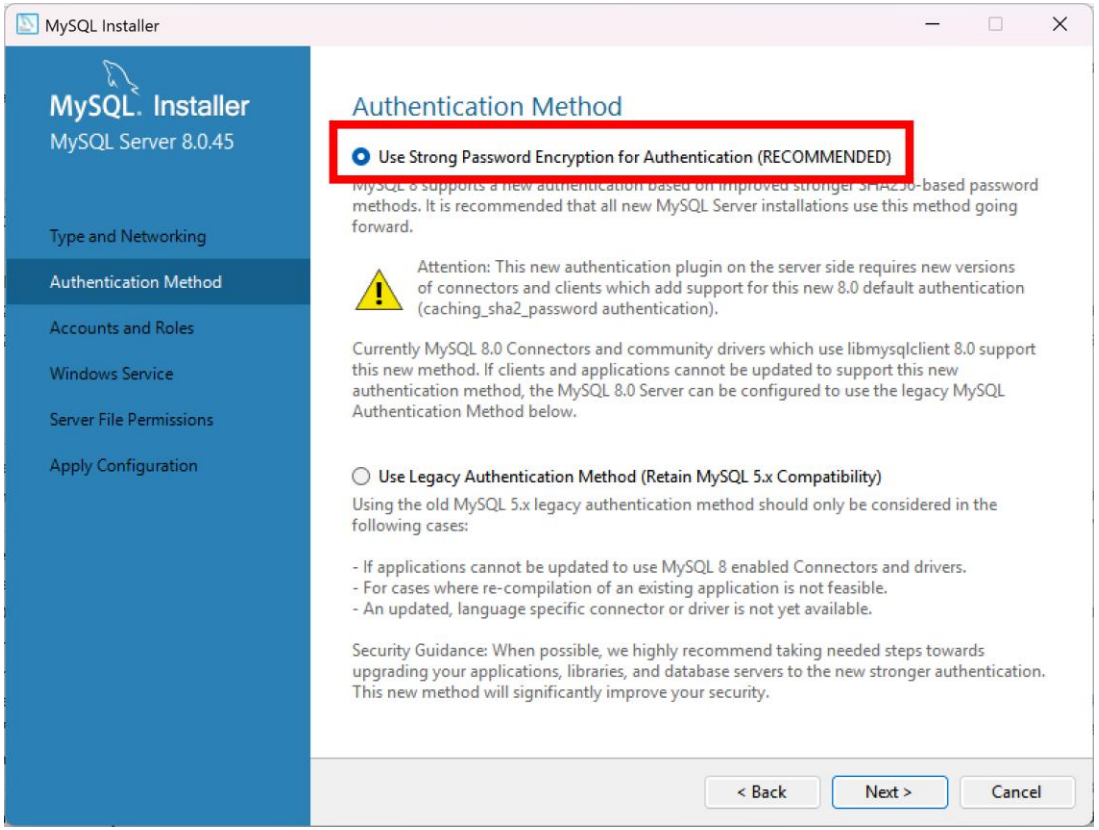




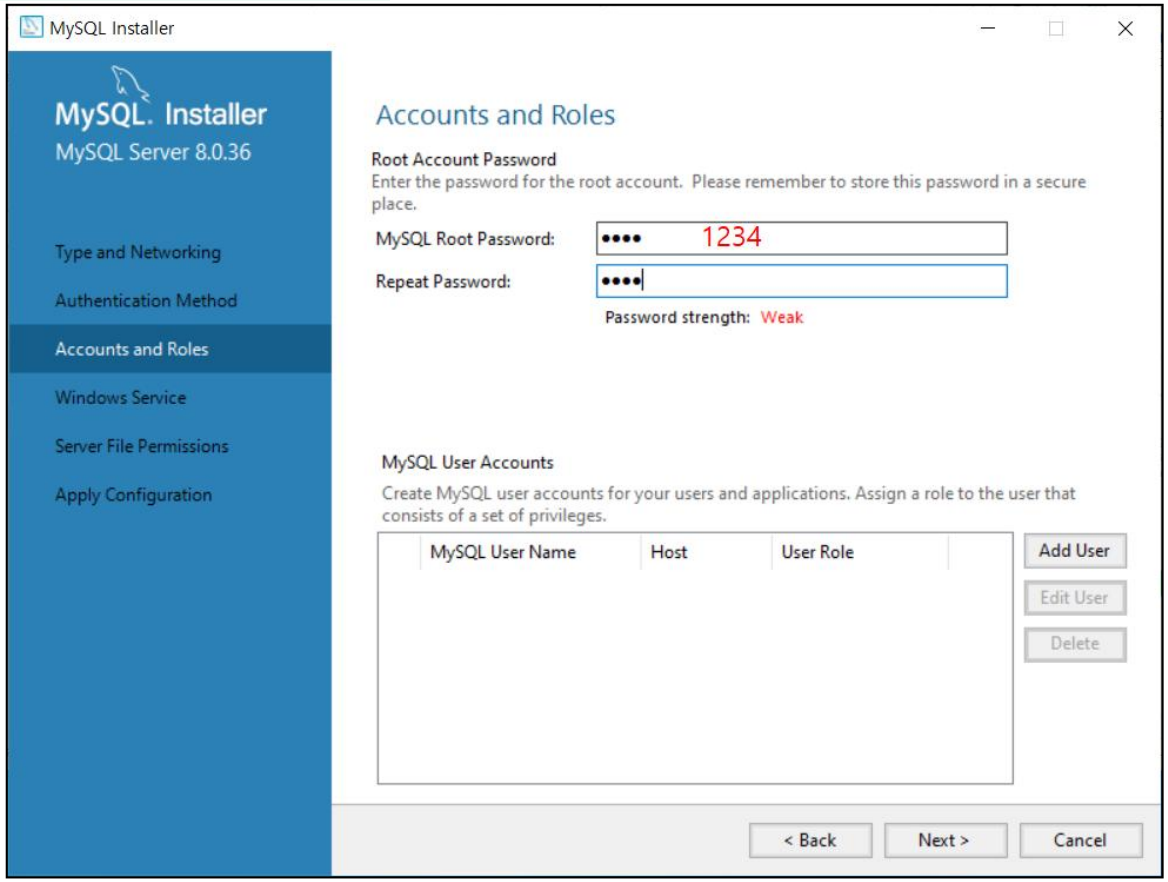
Server Type과 Networking 설정



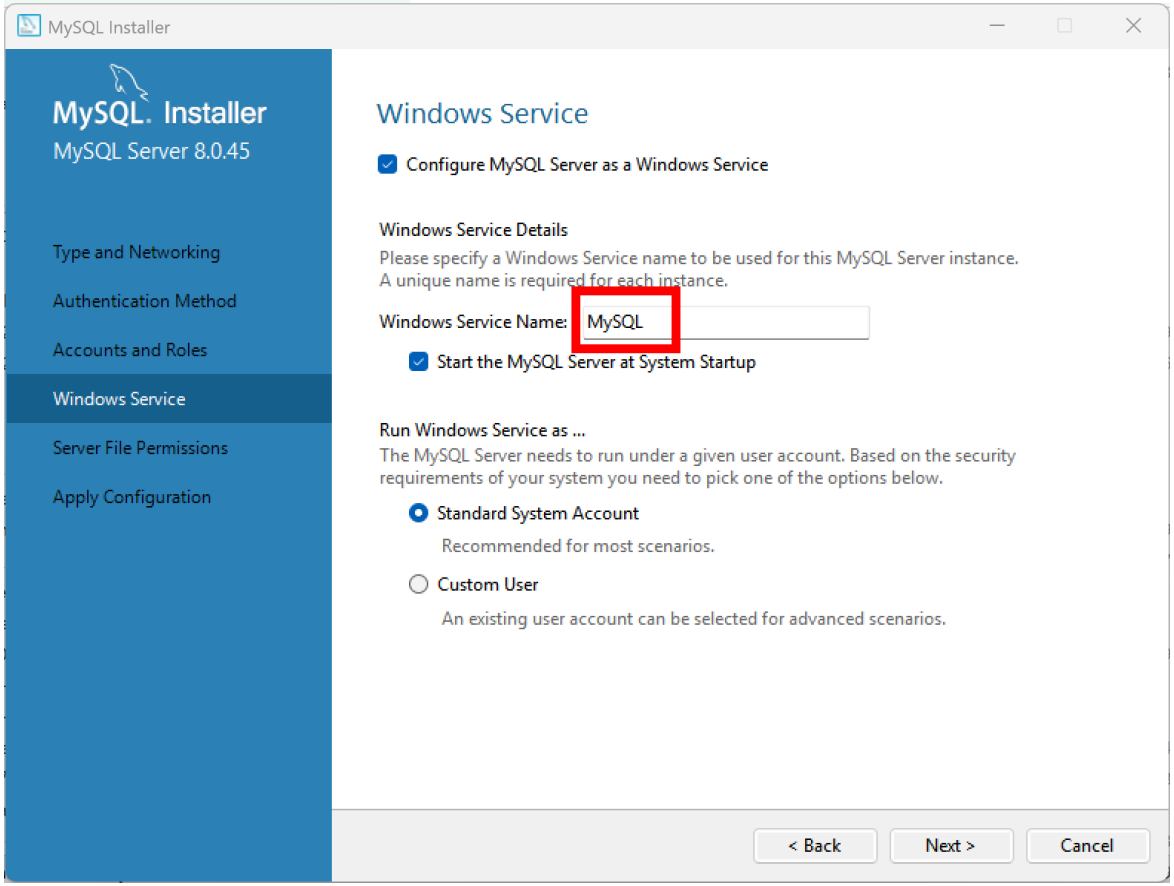
인증 방법 설정


MySQL Installer
MySQL. Installer
MySQL Server 8.0.45
Type and Networking
Authentication Method
Accounts and Roles
Windows Service
Server File Permissions
Apply Configuration
Authentication Method
Use Strong Password Encryption for Authentication (RECOMMENDED)
methods. It is recommended that all new MySQL Server installations use this method going forward.
Attention: This new authentication plugin on the server side requires new versions of connectors and clients which add support for this new 8.0 default authentication (caching_sha2_password authentication).
Currently MySQL 8.0 Connectors and community drivers which use libmysqlclient 8.0 support this new method. If clients and applications cannot be updated to support this new authentication method, the MySQL 8.0 Server can be configured to use the legacy MySQL Authentication Method below.
Use Legacy Authentication Method (Retain MySQL 5.x Compatibility)
Using the old MySQL 5.x legacy authentication method should only be considered in the following cases:
- If applications cannot be updated to use MySQL 8 enabled Connectors and drivers.
- For cases where re-compilation of an existing application is not feasible.
- An updated, language specific connector or driver is not yet available.
Security Guidance: When possible, we highly recommend taking needed steps towards upgrading your applications, libraries, and database servers to the new stronger authentication. This new method will significantly improve your security.
< Back
Next >
Cancel

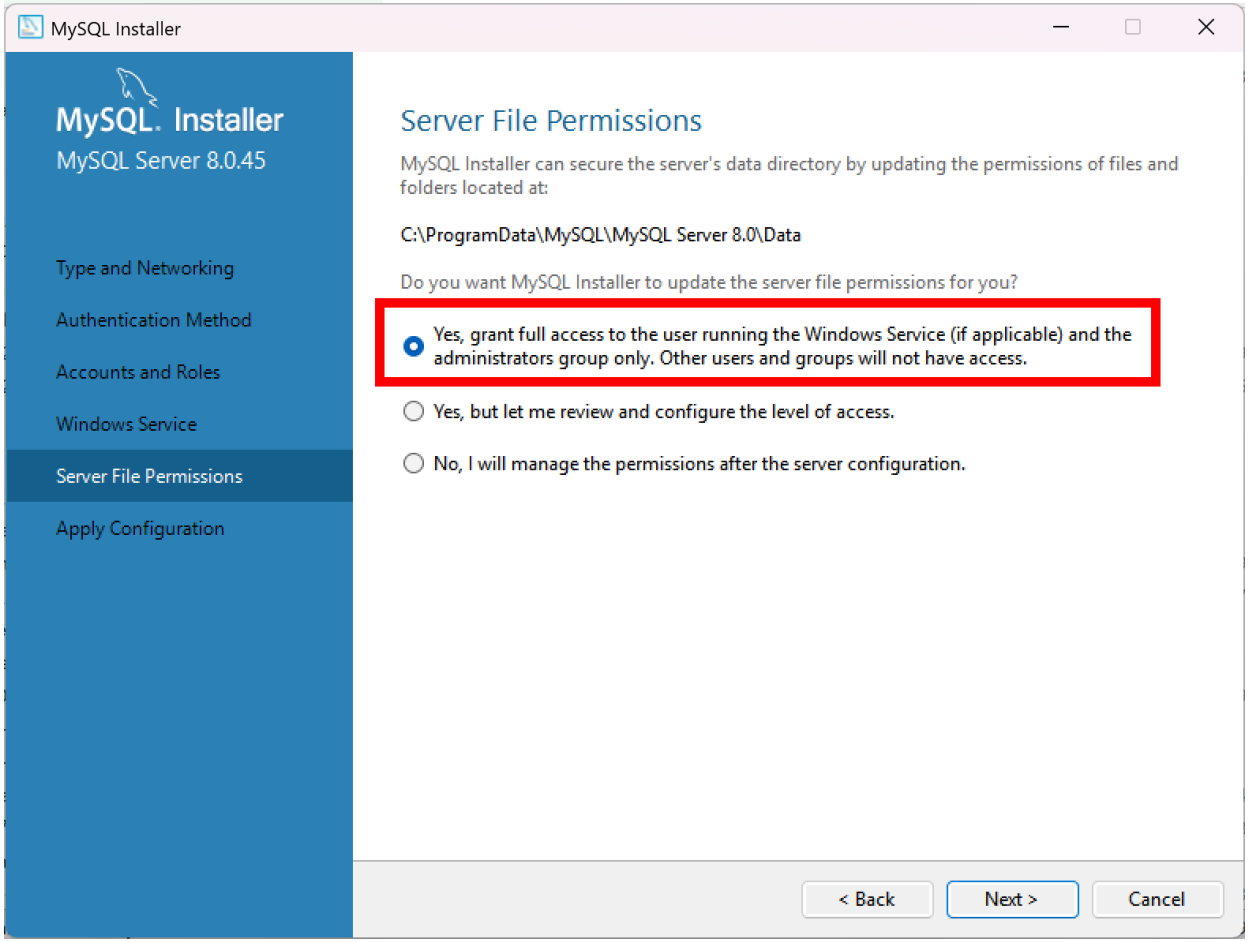
root 계정 비번 설정


MySQL Installer
MySQL. Installer
MySQL Server 8.0.36
Type and Networking
Authentication Method
Accounts and Roles
Windows Service
Server File Permissions
Apply Configuration
Accounts and Roles
Root Account Password
Enter the password for the root account. Please remember to store this password in a secure place.
MySQL Root Password:
1234
Repeat Password:
Password strength: Weak
MySQL User Accounts
Create MySQL user accounts for your users and applications. Assign a role to the user that consists of a set of privileges.
MySQL User Name
Host
User Role
Add User
Edit User
Delete
< Back
Next >
Cancel

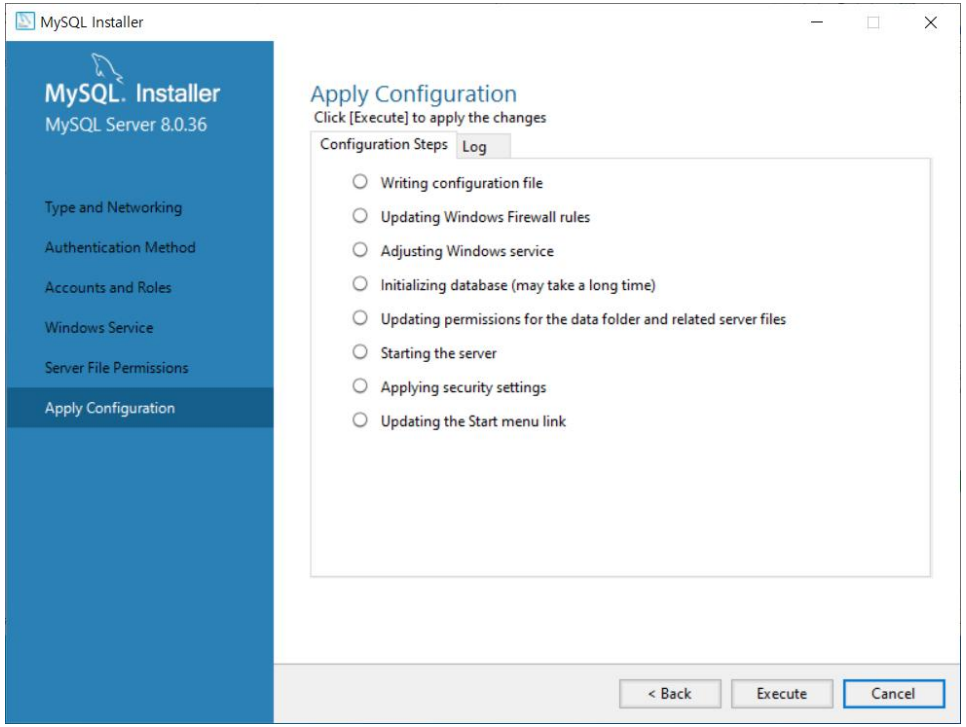
윈도우 서비스로 등록

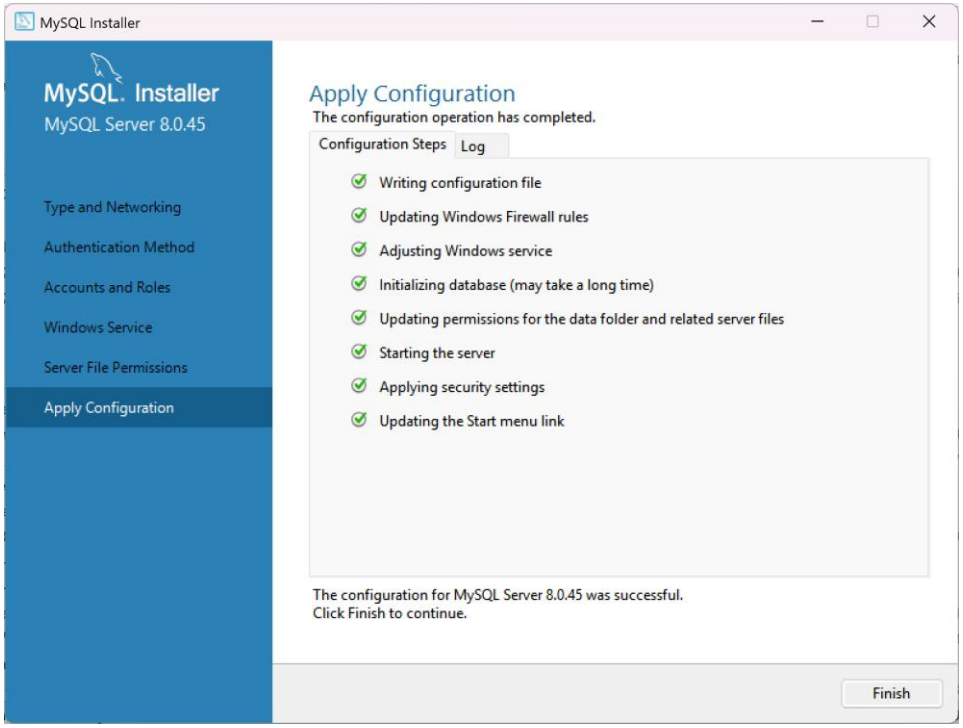

MySQL Installer
MySQL. Installer
MySQL Server 8.0.45
Type and Networking
Authentication Method
Accounts and Roles
Windows Service
Server File Permissions
Apply Configuration
Windows Service
Configure MySQL Server as a Windows Service
Windows Service Details
Please specify a Windows Service name to be used for this MySQL Server instance.
A unique name is required for each instance.
Windows Service Name: MySQL
Start the MySQL Server at System Startup
Run Windows Service as ...
The MySQL Server needs to run under a given user account. Based on the security requirements of your system you need to pick one of the options below.
Standard System Account
Recommended for most scenarios.
Custom User
An existing user account can be selected for advanced scenarios.
< Back
Next >
Cancel

파일 접근 권한 설정

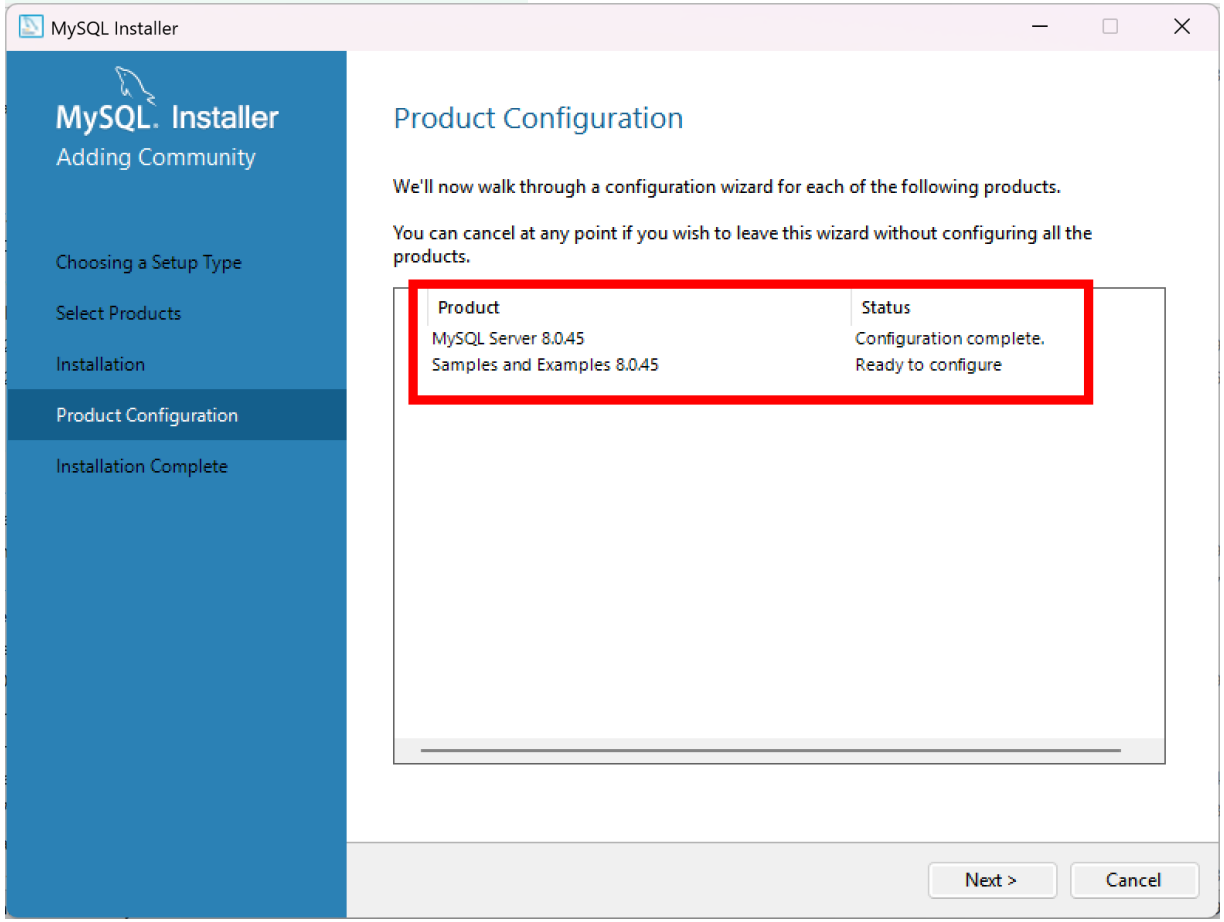

MySQL Installer
MySQL. Installer
MySQL Server 8.0.45
Type and Networking
Authentication Method
Accounts and Roles
Windows Service
Server File Permissions
Apply Configuration
Server File Permissions
MySQL Installer can secure the server's data directory by updating the permissions of files and folders located at:
C:\ProgramData\MySQL\MySQL Server 8.0\Data
Do you want MySQL Installer to update the server file permissions for you?
Yes, grant full access to the user running the Windows Service (if applicable) and the administrators group only. Other users and groups will not have access.
Yes, but let me review and configure the level of access.
No, I will manage the permissions after the server configuration.
< Back
Next >
Cancel

Configuration 적용

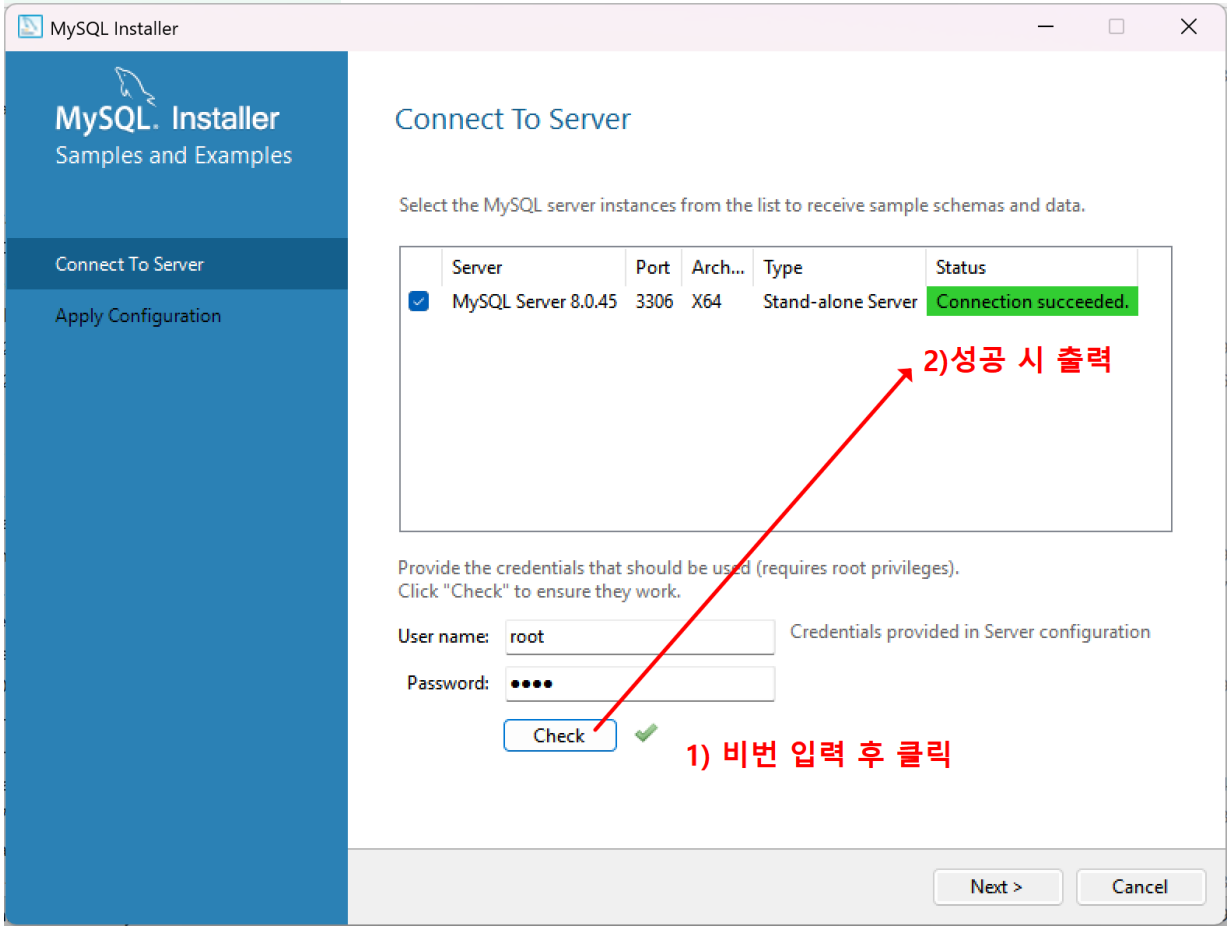




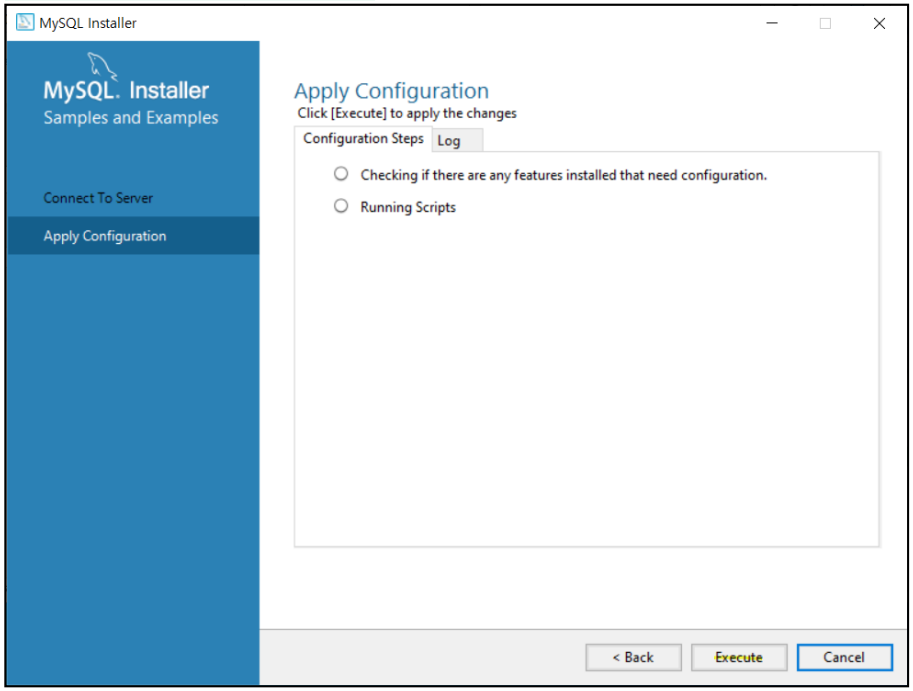
Product Configuration 적용

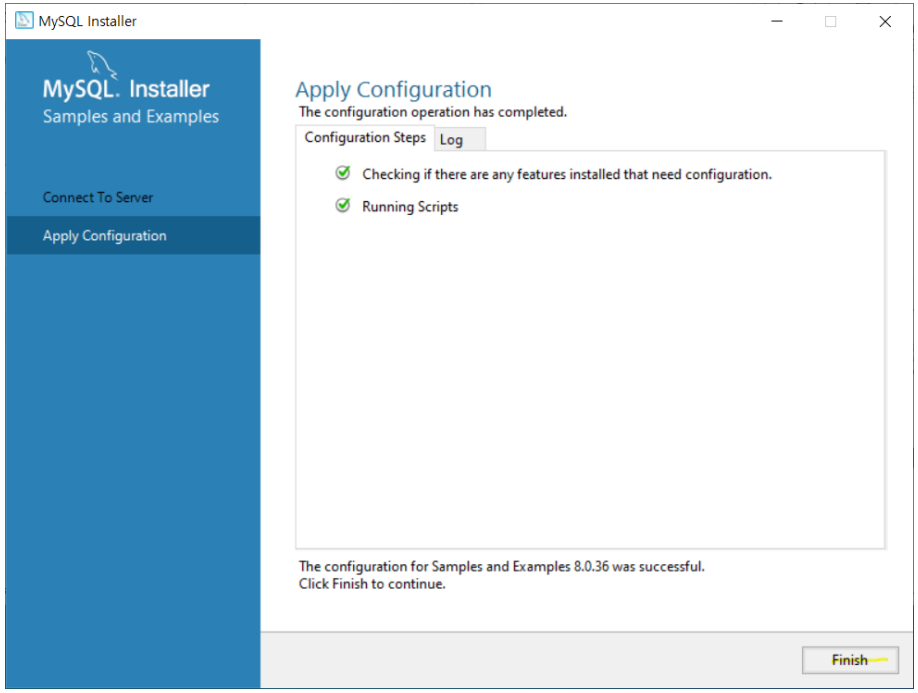


Server 연동 체크

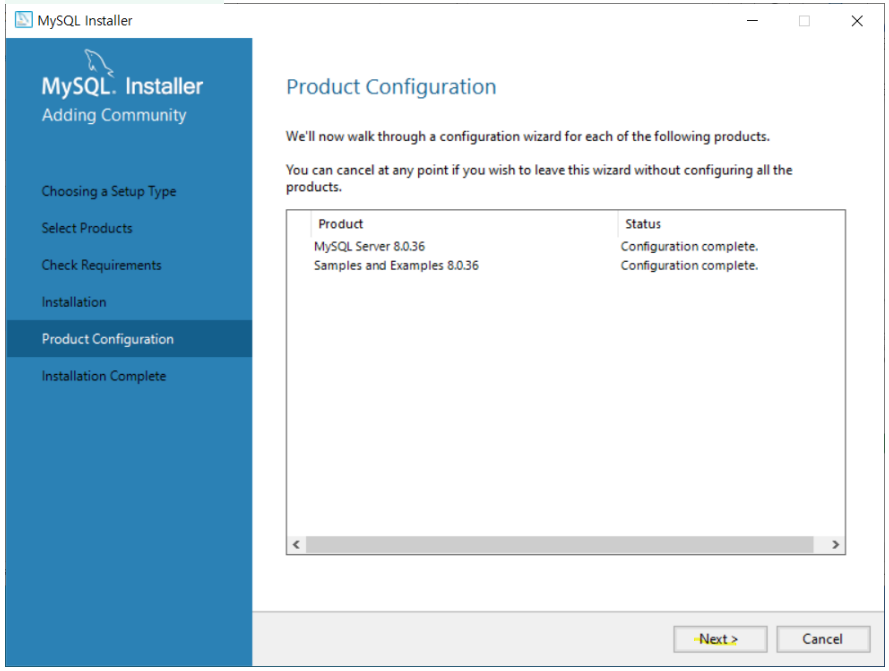

MySQL Installer
MySQL. Installer
Samples and Examples
Connect To Server
Apply Configuration
Connect To Server
Select the MySQL server instances from the list to receive sample schemas and data.
Server
Port
Arch...
Type
Status
MySQL Server 8.0.45
3306
X64
Stand-alone Server
Connection succeeded.
2)성공 시 출력
Provide the credentials that should be used (requires root privileges).
Click "Check" to ensure they work.
User name:
root
Credentials provided in Server configuration
Password:
Check
1) 비번 입력 후 클릭
Next >
Cancel

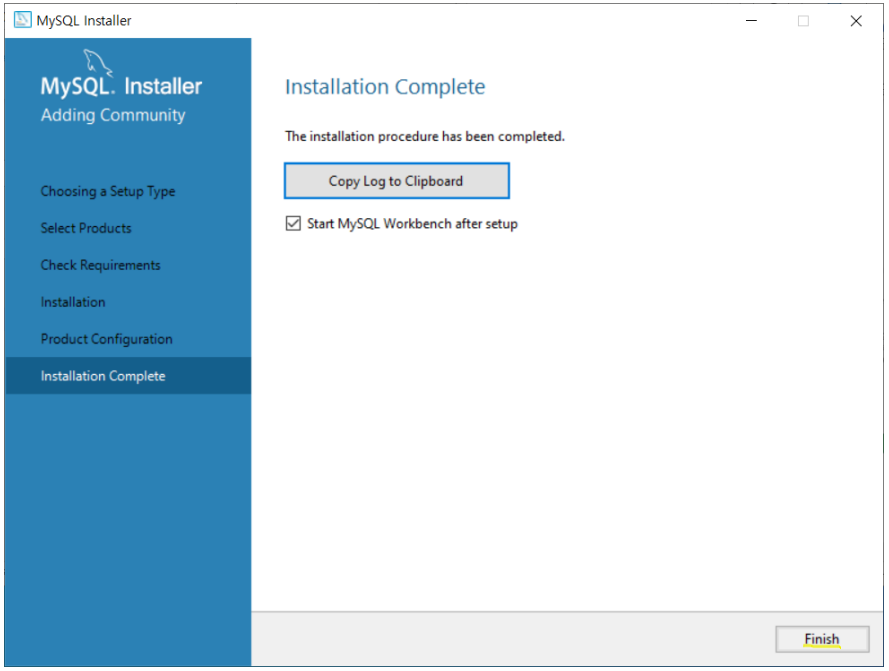
Configuration 설정



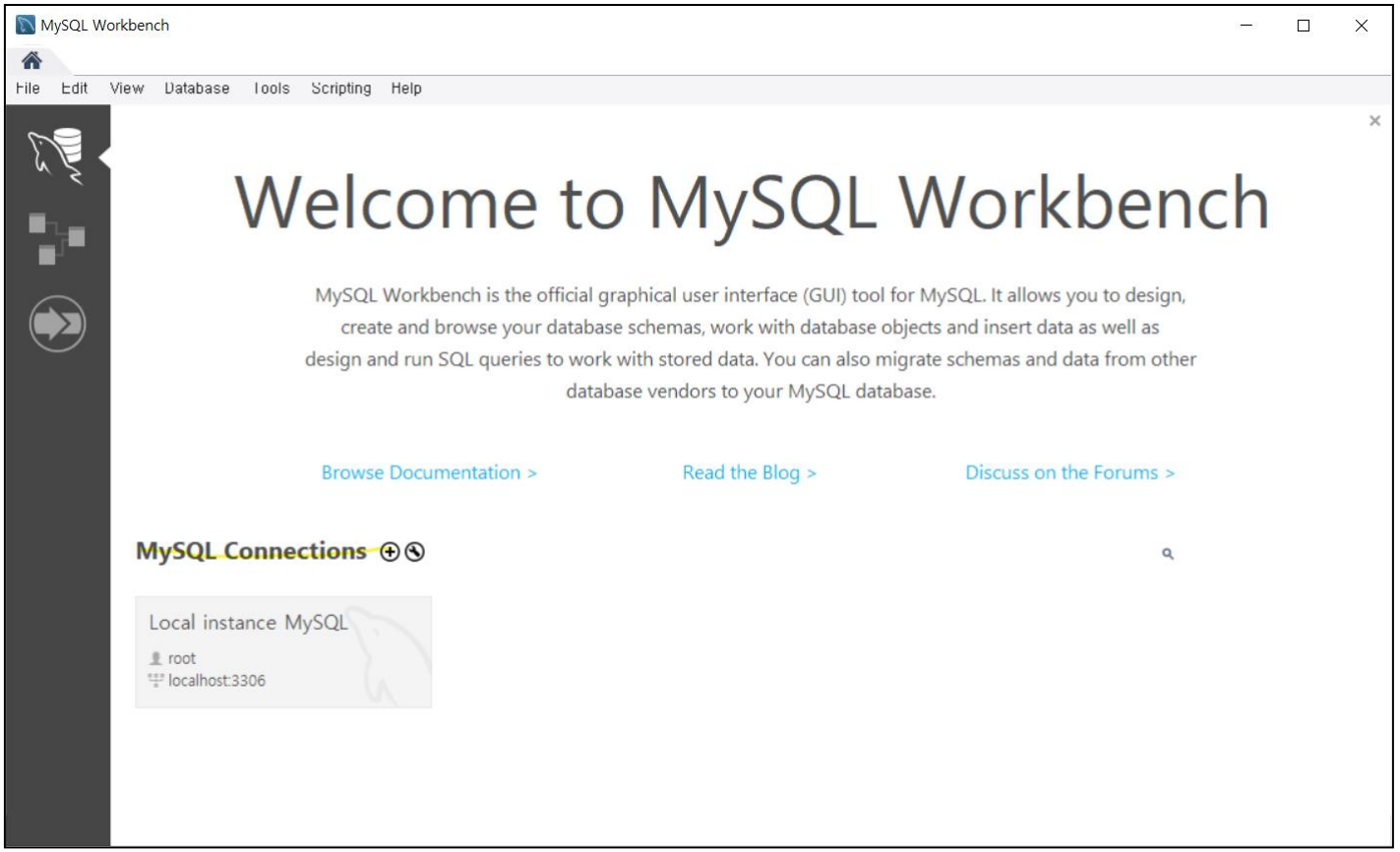


설치 완료

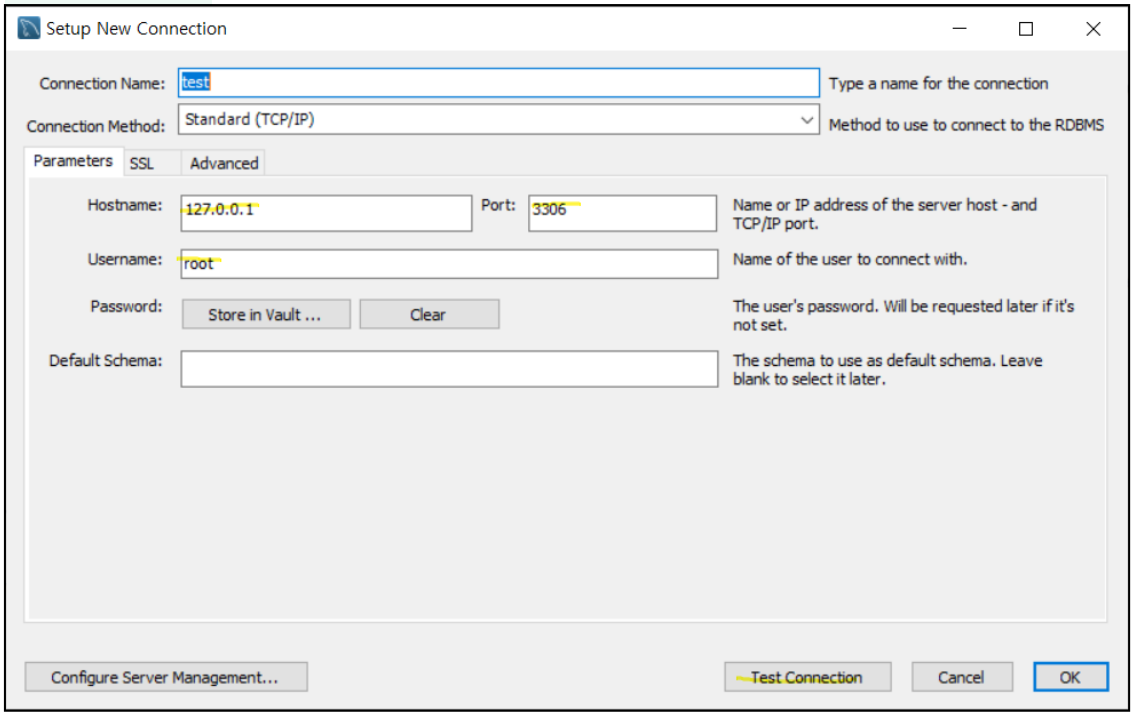

MySQL Installer
MySQL. Installer
Adding Community
Choosing a Setup Type
Select Products
Check Requirements
Installation
Product Configuration
Installation Complete
Product Configuration
We'll now walk through a configuration wizard for each of the following products.
You can cancel at any point if you wish to leave this wizard without configuring all the products.
Product
Status
MySQL Server 8.0.36
Configuration complete.
Samples and Examples 8.0.36
Configuration complete.
Next >
Cancel


MySQL Installer
MySQL. Installer
Adding Community
Choosing a Setup Type
Select Products
Check Requirements
Installation
Product Configuration
Installation Complete
Installation Complete
The installation procedure has been completed.
Copy Log to Clipboard
Start MySQL Workbench after setup
Finish

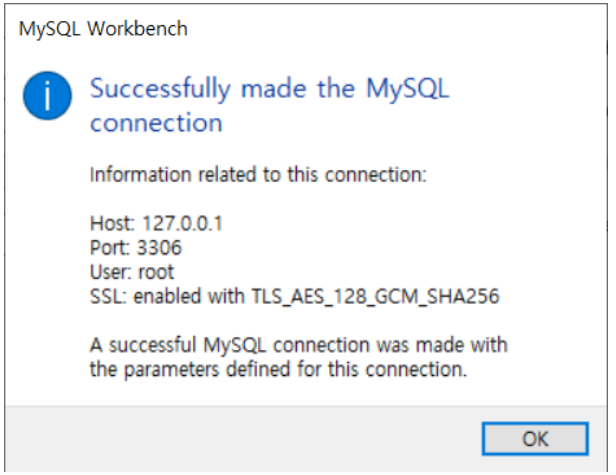
Workbench 시작



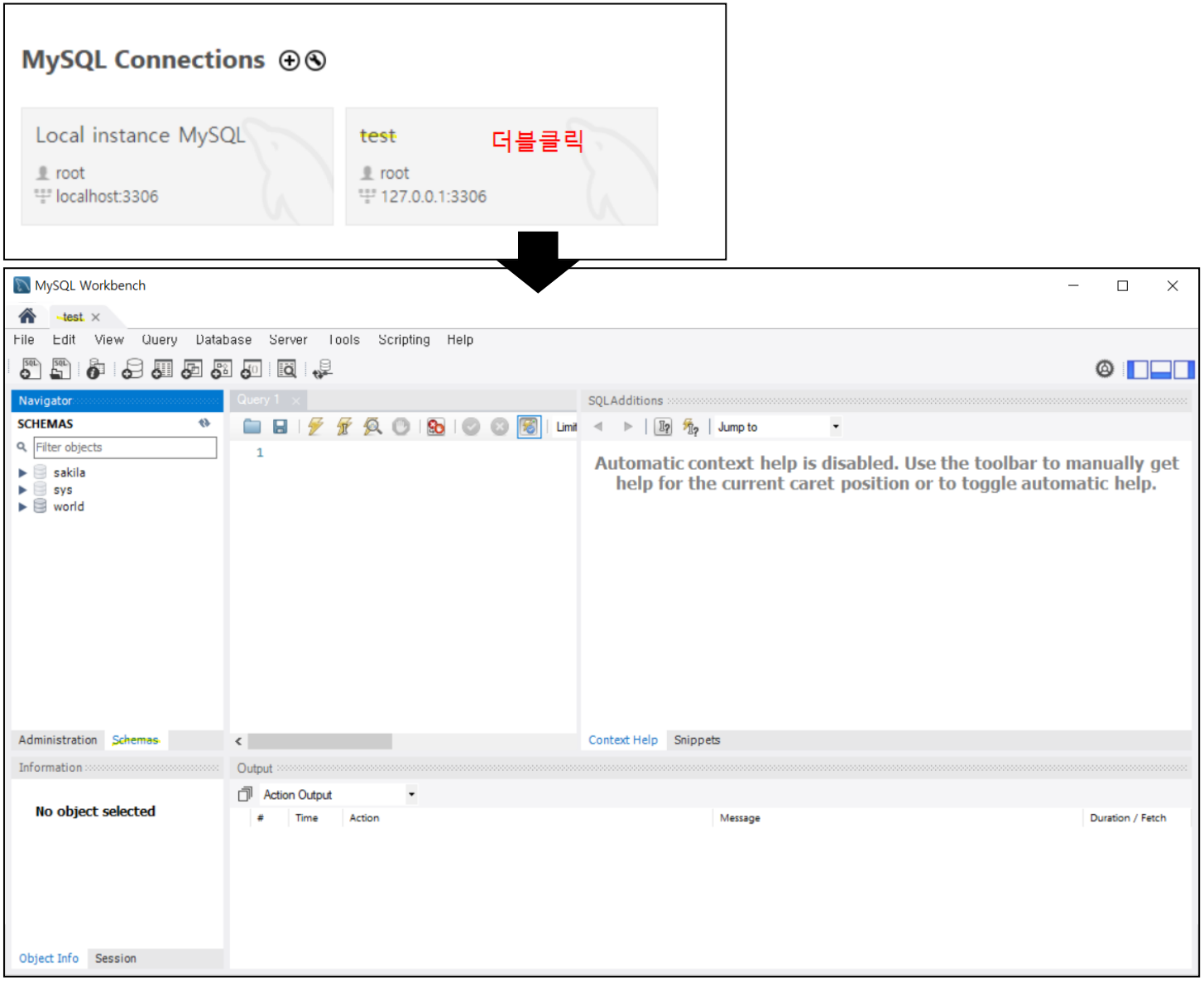
New 연결


Setup New Connection
Connection Name:
test
Type a name for the connection
Connection Method:
Standard (TCP/IP)
Method to use to connect to the RDBMS
Parameters
SSL
Advanced
Hostname:
127.0.0.1
Port:
3306
Name or IP address of the server host - and TCP/IP port.
Username:
root
Name of the user to connect with.
Password:
Store in Vault ...
Clear
The user's password. Will be requested later if it's not set.
Default Schema:
The schema to use as default schema. Leave blank to select it later.
Configure Server Management...
Test Connection
Cancel
OK

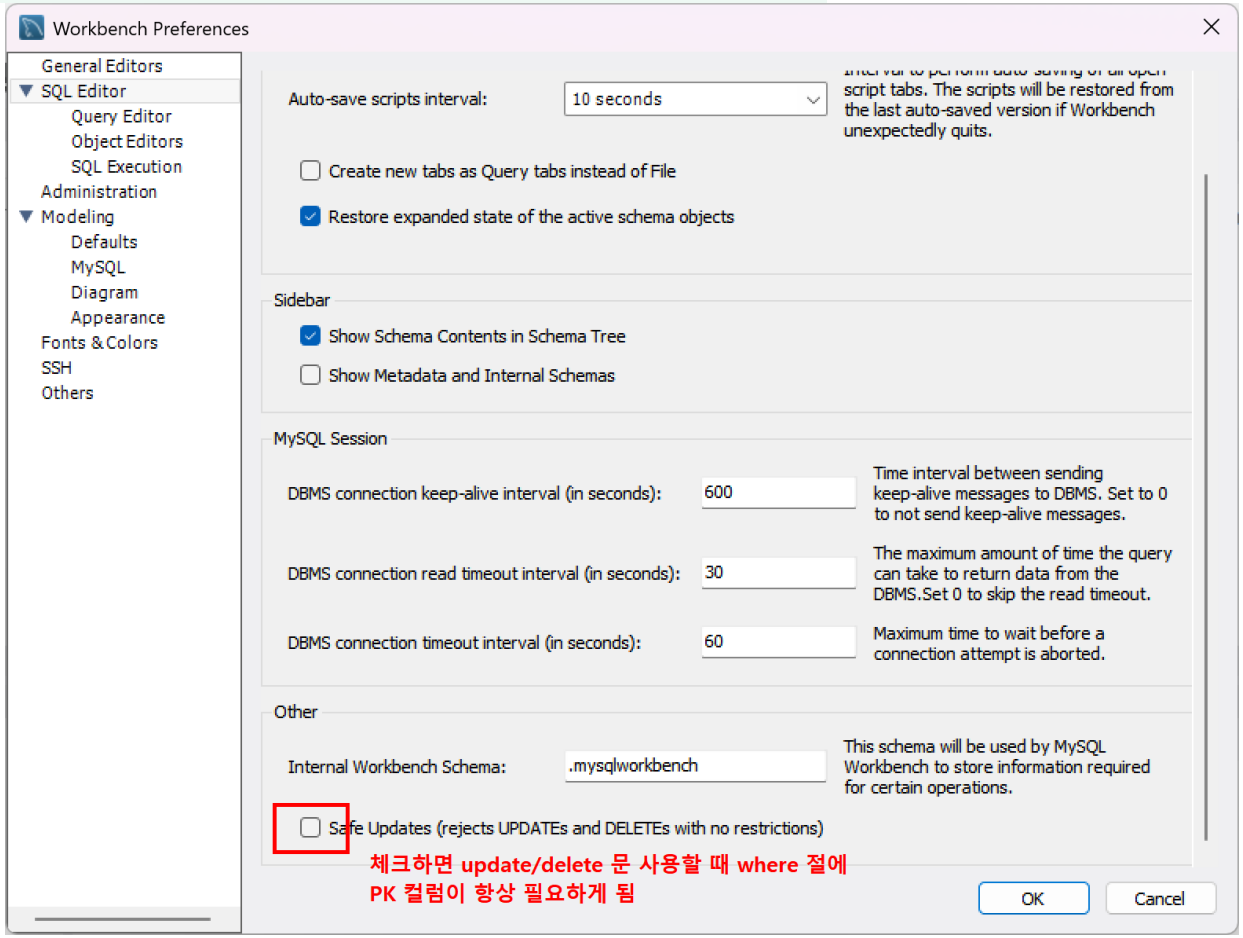

Connect to MySQL Server
Please enter password for the following service:
Service:
Mysql@127.0.0.1:3306
User:
root
Password:
****
1234
Save password in vault
OK
Cancel


MySQL Workbench
Successfully made the MySQL connection
Information related to this connection:
Host: 127.0.0.1
Port: 3306
User: root
SSL: enabled with TLS_AES_128_GCM_SHA256
A successful MySQL connection was made with the parameters defined for this connection.
OK

❑ MySQL8 설치

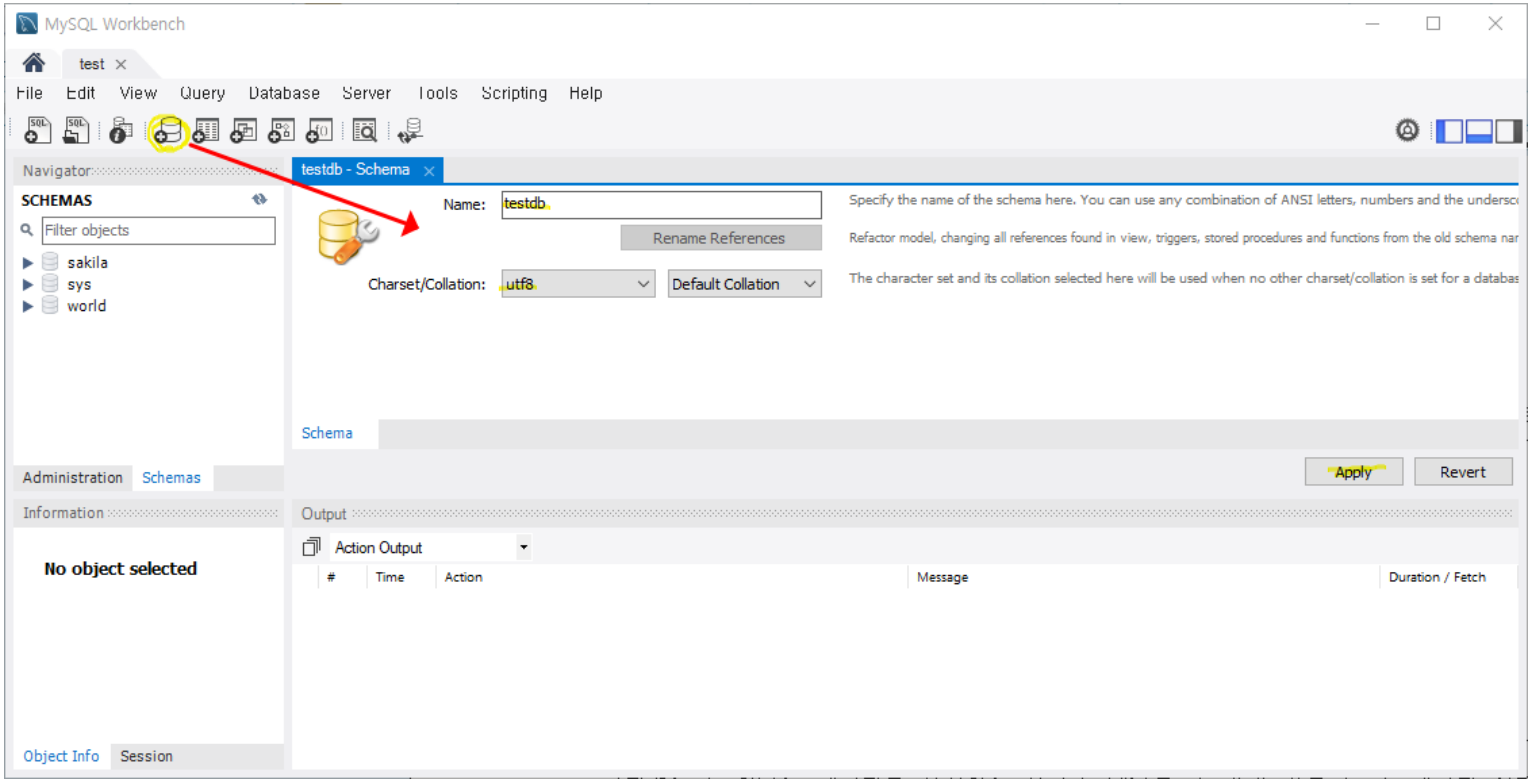

MySQL Connections
Local instance MySQL
root
localhost:3306
test
root
127.0.0.1:3306
더블클릭
MySQL Workbench
test
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
sakila
sys
world
Query 1
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Administration Schemas
Context Help Snippets
Information
No object selected
Object Info Session
Output
Action Output
Time Action Message Duration / Fetch

제약없이 update/delete 사용 가능하도록 설정



❑ SQL문 실습에 사용할 테이블 생성

testdb 데이터베이스 생성



❑ SQL문 실습에 사용할 테이블 생성

제공된 scott_mysql.sql 사용하여 실습 테이블 생성하기

```
# scott_mariadb.sql
CREATE TABLE IF NOT EXISTS DEPT
 (DEPTNO INT  PRIMARY KEY,
   DNAME VARCHAR(14) ,
   LOC VARCHAR(13) ) ;

CREATE TABLE emp
 (EMPNO INT,
   ENAME VARCHAR(10),
   JOB VARCHAR(9),
   MGR INT,
   HIREDATE DATE,
   SAL FLOAT,
   COMM INT,
   DEPTNO INT,
   PRIMARY KEY(EMPNO),
   FOREIGN KEY (DEPTNO)  REFERENCES DEPT(DEPTNO) );

CREATE TABLE SALGRADE
      ( GRADE INT,
   LOSAL INT,
   HISAL INT );


INSERT INTO DEPT VALUES
   (10,'ACCOUNTING','NEW YORK');
INSERT INTO DEPT VALUES (20,'RESEARCH','DALLAS');
INSERT INTO DEPT VALUES
   (30,'SALES','CHICAGO');
INSERT INTO DEPT VALUES
   (40,'OPERATIONS','BOSTON');
INSERT INTO EMP VALUES
(7369,'SMITH','CLERK',7902,STR_TO_DATE('17-12-1980','%d-%m-%Y'),800,NULL,20);
INSERT INTO EMP VALUES
(7499,'ALLEN','SALESMAN',7698,STR_TO_DATE('20-2-1981','%d-%m-%Y'),1600,300,30);
INSERT INTO EMP VALUES
(7521,'WARD','SALESMAN',7698,STR_TO_DATE('22-2-1981','%d-%m-%Y'),1250,500,30);
```

❑ SQL문 실습에 사용할 테이블 생성

실습에서 사용할 테이블은 다음과 같은 3가지 테이블이다.

emp 테이블(사원 테이블)

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	80/12/17	800		20
7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
7521	WARD	SALESMAN	7698	81/02/22	1250	500	30
7566	JONES	MANAGER	7839	81/04/02	2975		20
7654	MARTIN	SALESMAN	7698	81/09/28	1250	1400	30
7698	BLAKE	MANAGER	7839	81/05/01	2850		30
7782	CLARK	MANAGER	7839	81/06/09	2450		10
7788	SCOTT	ANALYST	7566	87/04/19	3000		20
7839	KING	PRESIDENT		81/11/17	5000		10
7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
7876	ADAMS	CLERK	7788	87/05/23	1100		20
7900	JAMES	CLERK	7698	81/12/03	950		30
7902	FORD	ANALYST	7566	81/12/03	3000		20
7934	MILLER	CLERK	7782	82/01/23	1300		10

다음과 같이 총 8개의 컬럼으로 구성되어 있다.

-사원번호(empno)
-사원명 (ename)
-직급 (job)
-관리자 번호 (mgr)
-입사일 (hiredate)
-급여 (sal)
-커미션(comm)
-부서번호 (deptno)

❑ SQL문 실습에 사용할 테이블 정보

dept 테이블 (부서 테이블)

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

부서 테이블 이름은 dept 이며 다음과 같이 총 3개의 컬럼으로 구성되어 있다.

-부서번호 (deptno)
-부서명 (dname)
-부서위치 (loc)

salgrade 테이블 (sal 등급 테이블)

GRADE	LOSAL	HISAL
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999

급여등급 테이블 이름은 salgrade 이며 다음과 같이 총 3개의 컬럼으로 구성되어 있다.

-등급 (grade)
-최소월급 (losal)
-최대월급 (hisal)

chapter 02.

DB 개요

- 데이터베이스 (DataBase: DB)
  개인, 회사, 관공서 등에서 업무적으로 필요한 데이터 의미.

- 데이터베이스 관리 시스템 (DataBase Management System : DBMS)
  -DataBase(데이터)를 전문적으로 관리해주는 S/W 임.
  -관리 방법 ? CRUD, 사용자 생성, Role 부여 등

- 데이터 저장 방법 ?
  -관계형 데이터베이스
  -객체지향 데이터베이스
  -망형 데이터베이스
  - …

❑ 관계형 데이터베이스

- 관계형 데이터베이스 (Relational DataBase : RDB)
  행(레코드)과 열(컬럼)으로 구성된 2차원 테이블 형식으로 데이터를 관리.

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	80/12/17	800		20
7499	ALLEN	SALESMAN	7698	81/02/20	1600	300	30
7521	WARD	SALESMAN	7698	81/02/22	1250	500	30
7566	JONES	MANAGER	7839	81/04/02	2975		20
7654	MARTIN	SALESMAN	7698	81/09/28	1250	1400	30
7698	BLAKE	MANAGER	7839	81/05/01	2850		30
7782	CLARK	MANAGER	7839	81/06/09	2450		10
7788	SCOTT	ANALYST	7566	87/04/19	3000		20
7839	KING	PRESIDENT		81/11/17	5000		10
7844	TURNER	SALESMAN	7698	81/09/08	1500	0	30
7876	ADAMS	CLERK	7788	87/05/23	1100		20
7900	JAMES	CLERK	7698	81/12/03	950		30
7902	FORD	ANALYST	7566	81/12/03	3000		20
7934	MILLER	CLERK	7782	82/01/23	1300		10

- 종류
  - MySQL
  - Oracle
  - MariaDB
  - MS-SQL
  - ...

- SQL (Structured Query Language)
  RDB에서 사용하는 프로그램 언어로서 대소문자 구별 안함.

- 종류1

  가. ANSI SQL
  -표준화된 SQL 의미
  -DBMS에 독립적
  모든 DBMS에서 사용가능한 문법 의미.

  나. 비 ANSI SQL
  -DBMS에 의존적.
  특정 DBMS에서만 사용가능한 문법 의미.

- 종류2

가. 데이터 조회
- DQL (Data Query Language, 데이터 쿼리어)라고 부름
- select 문 이용

나. 데이터 생성/수정/삭제
- DML (Data Manipulation Language, 데이터 조작어)라고 부름.
- 레코드 생성 : insert 문
- 레코드 수정 : update 문
- 레코드 생성 : delete 문

다. 테이블 생성/수정/삭제
- DDL (Data Definition Language, 데이터 정의어)라고 부름.
- 테이블 생성 : create 문
- 테이블 수정 : alter 문
- 테이블 생성 : drop 문

라. 트랜잭션 (Transaction)

-TCL (Transaction Control Language, 트랜잭션 관리)라고 부름.

- 트랜잭션 적용 : commit 문
- 트랜잭션 취소 : rollback 문

chapter 03.

SELECT

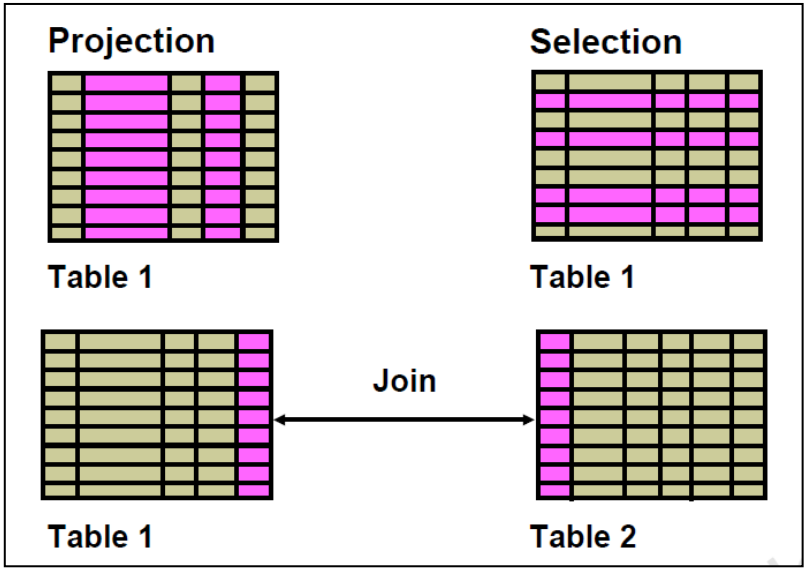
❑ SELECT 개요

▪ SELECT 기능
: 데이터베이스로부터 데이터를 검색하는 기능을 갖는다.

- selection : 질의에 대해 테이블의 행을 선택하기 위해 사용.
- projection : 질의에 대해 테이블의 열을 선택하기 위해 사용.
- join : 여러 테이블이 공통적으로 가진 컬럼을 이용해서 다른 테이블에 저장되어 있는 데이터를 한번에 가져오기 위해 사용.

▪ 기본적인 SELECT 문법

```
SELECT [DISTINCT] { *, column [alias],... }
[FROM    table];
[limit len| start, len]
```



▪ 테이블내의 모든 데이터 보기 (* 는 모든 컬럼 의미)

```
SELECT * FROM dept;
```

Result Grid | Filter Rows:

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON
NULL	NULL	NULL

```
select * from emp limit 3;
```

Result Grid | Filter Rows: | Edit: | Export/Imp

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

▪ 테이블내의 특정 컬럼 데이터 보기

- SELECT 뒤에 해당 컬럼을 차례대로 기술, 쉼표로 구분해서 여러 개 지정 가능.

```
SELECT empno, ename, job, hiredate
FROM emp;
```

Result Grid | Filter Rows:

empno	ename	job	hiredate
7369	SMITH	CLERK	1980-12-17
7499	ALLEN	SALESMAN	1981-02-20
7521	WARD	SALESMAN	1981-02-22
7566	JONES	MANAGER	1981-04-02
7654	MARTIN	SALESMAN	1981-09-28
7698	BLAKE	MANAGER	1981-05-01
7782	CLARK	MANAGER	1981-06-09
7788	SCOTT	ANALYST	1987-06-28
7839	KING	PRESIDENT	1981-11-17

❑ 산술연산을 이용한 SQL 문

- SQL 문장내의 숫자 타입에는 + , – , * , /, % 사용 가능하다.
- 연산식이 컬럼명으로 표시된다.

```
SELECT empno, ename,   sal,  sal + 10, sal-10, sal*1.1 , sal / 3,
sal % 3,  mod(sal, 3) FROM emp;
```

empno	ename	sal	sal + 10	sal-10	sal*1.1	sal / 3	sal % 3	mod(sal, 3)
7369	SMITH	800	810	790	880.0000000000001	266.6666666666667	2	2
7499	ALLEN	1600	1610	1590	1760.0000000000002	533.3333333333334	1	1
7521	WARD	1250	1260	1240	1375	416.6666666666667	2	2
7566	JONES	2975	2985	2965	3272.5000000000005	991.6666666666666	2	2

- 컬럼에 별칭(alias) 사용.

```
SELECT empno AS "사원 번호" , ename AS 성명, sal 급여
FROM emp;
```

Result Grid | Filter Rows:

사원 번호	성명	급여
7369	SMITH	800
7499	ALLEN	1600
7521	WARD	1250

❑ null 값

- 이용 불가능한(unavailable), 비교 자체가 불가능한
- 지정되지 않은(unassigned)
- 알 수 없는 (unknown)
- 적용할 수 없는(Inapplicable) 값을 의미한다.

컬럼에 기본적으로 null 값을 허용하며 제약조건을 이용해서 null 값을 허용하지 않을 수도 있다.
주의할 점은 null 값의 연산결과는 null 값으로 나온다는 것이다.
null 값의 비교는 IS NULL, IS NOT NULL 이라는 정해진 문구를 사용해야 제대로 된 결과를 얻을 수 있다.

```
SELECT empno, ename, sal, comm, comm + 10, (sal * 12) + comm as "연봉"
FROM emp;
```

Result Grid | Filter Rows: | Export:

empno	ename	sal	comm	comm + 10	연봉
7369	SMITH	800	NULL	NULL	NULL
7499	ALLEN	1600	300	310	19500
7521	WARD	1250	500	510	15500
7566	JONES	2975	NULL	NULL	NULL

null 값을 가진 컬럼을 연산하기 위해서는 IFNULL 함수를 사용할 수 있다.
IFNULL 함수는 컬럼값이 null인 경우에 기본값으로 설정한다.

IFNULL (컬럼명, 기본값)

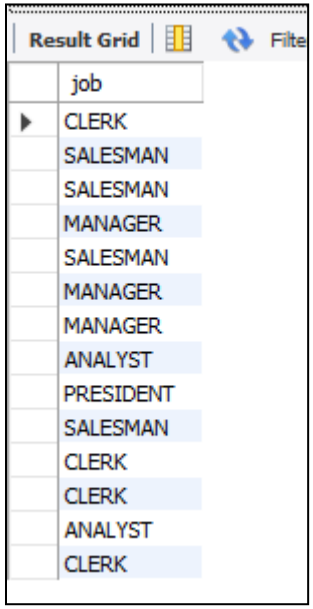
```
SELECT empno, ename,   sal,  comm,  (sal * 12) + comm as "연봉",
(sal*12) + ifnull(comm, 0) as "연봉"
FROM emp;
```

empno	ename	sal	comm	연봉	연봉
7369	SMITH	800	NULL	NULL	9600
7499	ALLEN	1600	300	19500	19500
7521	WARD	1250	500	15500	15500
7566	JONES	2975	NULL	NULL	35700

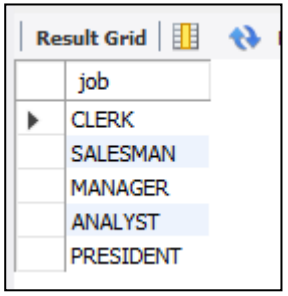
❑ 중복 행 제거

- DISTINCT 키워드 이용
  중복된 값 제거하여 한번만 출력.

```
SELECT job
FROM emp;
```




```
SELECT distinct job
FROM emp;
```



- WHERE 기능
  - 테이블내의 모든 행을 검색하는 대신 검색 조건을 지정하여 사용자가 원하는 행들만 검색.

- 기본적인 SELECT ~ WHERE 문법

```
SELECT [DISTINCT] { *, column [alias],... }
[FROM    table]
[WHERE 조건식];
```

```
SELECT empno, ename, job, deptno
FROM emp
WHERE deptno = 30;
```

Result Grid | Filter Rows:

empno	ename	job	deptno
7499	ALLEN	SALESMAN	30
7521	WARD	SALESMAN	30
7654	MARTIN	SALESMAN	30
7698	BLAKE	MANAGER	30
7844	TURNER	SALESMAN	30
7900	JAMES	CLERK	30

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/non-typed-operators.html

14.4 Operators

14.4.1 Operator Precedence
14.4.2 Comparison Functions and Operators
14.4.3 Logical Operators
14.4.4 Assignment Operators

Table 14.3 Operators

Name	Description
&	Bitwise AND
>	Greater than operator
>>	Right shift
>=	Greater than or equal operator
<	Less than operator
<>, !=	Not equal operator
<<	Left shift
<=	Less than or equal operator
<=>	NULL-safe equal to operator
%, MOD	Modulo operator
*	Multiplication operator
+	Addition operator
-	Minus operator
-	Change the sign of the argument
AND, &&	Logical AND
BETWEEN ... AND ...	Whether a value is within a range of values
BINARY	Cast a string to a binary string	8.0.27
CASE	Case operator
DIV	Integer division
IN()	Whether a value is within a set of values
IS	Test a value against a boolean
IS NOT	Test a value against a boolean
IS NOT NULL	NOT NULL value test
IS NULL	NULL value test
LIKE	Simple pattern matching
MEMBER OF()	Returns true (1) if first operand matches any element of JSON array passed as second operand, otherwise returns false (0)	8.0.17
NOT, !	Negates value
NOT BETWEEN ... AND ...	Whether a value is not within a range of values
NOT IN()	Whether a value is not within a set of values
NOT LIKE	Negation of simple pattern matching
NOT REGEXP	Negation of REGEXP
OR, ||	Logical OR
REGEXP	Whether string matches regular expression
RLIKE	Whether string matches regular expression

❑ WHERE – 비교 연산자 1

연산자	의미
=	같다
>	보다 크다
>=	보다 크거나 같다
<	보다 작다
<=	보다 작거나 같다
!=	같지 않다

```
SELECT empno, ename, sal
FROM emp
WHERE sal <= 1000;
```

Result Grid | Filter Rows

empno	ename	sal
7369	SMITH	800
7900	JAMES	950

```
SELECT empno, ename, job, deptno
FROM emp
WHERE job = 'salesman'
```

Result Grid | Filter Rows:

empno	ename	job	deptno
7499	ALLEN	SALESMAN	30
7521	WARD	SALESMAN	30
7654	MARTIN	SALESMAN	30
7844	TURNER	SALESMAN	30

```
SELECT empno, ename, job, deptno ,
sal, hiredate
FROM emp
WHERE hiredate = '1980-12-17';
```

❑ WHERE – 비교 연산자 2

연산자	의미
BETWEEN a AND b	a와 b 두 값의 범위에 포함되는
IN(값, 값, ..)	괄호안의 값과 일치하는
LIKE '%|_'	지정된 값이 포함되는 (%와 _ 같이 사용)
IS NULL	null 값 조회

```
SELECT empno, ename, job, deptno
FROM emp
WHERE sal BETWEEN 800 AND 2000;
```

```
SELECT empno, ename, sal, comm
FROM emp
WHERE comm IS NULL;
```

```
SELECT empno, ename, sal
FROM emp
WHERE empno IN (7369,7566,7698);
```

❑ WHERE – 비교 연산자 2

• LIKE 연산자
  –검색하고자 하는 문자열을 정확히 알 수 없는 경우에 사용.
  –패턴 매칭 연산자 이용 (와일드 카드)

기호	의미
%	0 글자 이상의 임의 문자 치환
_	1 글자의 임의 문자 치환

```
SELECT empno, ename, sal
FROM emp
WHERE ename LIKE 'A%';
```

```
SELECT empno, ename, sal
FROM emp
WHERE ename LIKE '%T%';
```

```
SELECT empno, ename, sal
FROM emp
WHERE ename LIKE '_L%';
```

– 검색하고자 하는 문자열에 패턴 매칭 연산자가 포함되어 있을 때는 ESCAPE 옵션을 사용한다.

```
SELECT empno, ename, sal
FROM emp
WHERE ename LIKE '%E_%' ESCAPE 'E';
```

❑ WHERE – 논리 연산자

- WHERE 절에 부여할 조건이 여러 개인 경우에 사용한다.

연산자	의미
AND	두 개의 조건이 모두 TRUE인 경우에만 TRUE 리턴
OR	두 개의 조건 중 하나만 TRUE라도 TRUE 리턴
NOT	FALSE 이면 TRUE 리턴

```
SELECT empno, ename, job, deptno, hiredate
FROM emp
WHERE job = 'salesman' AND sal >= 1500;
```

```
SELECT empno, ename, job, deptno, hiredate
FROM emp
WHERE job = 'salesman' OR sal >= 1500;
```

```
SELECT empno, ename, sal, comm
FROM emp
WHERE comm IS NOT NULL;
```

❑ WHERE – 부정 연산자

- WHERE 절에 사용 가능한 부정 연산자.

연산자	의미
!=	같지 않다
NOT 컬럼명 =	~와 같지 않다
NOT 컬럼명 >	~ 보다 크지 않다
NOT BETWEEN a AND b	a와 b 값 사이에 있지 않다
NOT IN (값, 값, ..)	값 들과 일치하지 않다
IS NOT NULL	NULL 값을 갖지 않다
NOT LIKE	패턴과 일치하지 않다

```
SELECT empno, ename, job, deptno
FROM emp
WHERE sal NOT BETWEEN 800 AND 2000;
```

```
SELECT empno, ename, sal
FROM emp
WHERE empno NOT IN (7369,7566,7698);
```

```
SELECT empno, ename, sal
FROM emp
WHERE ename NOT LIKE 'A%';
```

❑ 정렬

- SELECT 문장에 의해 검색된 결과를 정렬 (기본은 오름차순, 다중 정렬 가능)

```
SELECT [DISTINCT] { *, column [alias],... }
[FROM    table]
[WHERE 조건식]
[ORDER BY {column|exp }  [ASC|DESC] ];
```

- null
  - MySQL은 null값을 가장 작은 값으로 간주한다.

```
SELECT empno, ename,  hiredate
FROM emp
ORDER BY hiredate;
```

```
SELECT empno, ename,  hiredate
FROM emp
ORDER BY hiredate DESC ;
```

```
SELECT empno, ename,  sal * 12 AS Annual
FROM emp
ORDER BY Annual;
```

```
SELECT empno, ename,  sal * 12 AS Annual
FROM emp
ORDER BY 3;
```

```
SELECT empno, ename,  sal * 12 AS Annual
FROM emp
ORDER BY Annual, empno DESC;
```

```
SELECT empno, ename,  sal * 12 AS Annual
FROM emp
ORDER BY 3, 1 DESC;
```

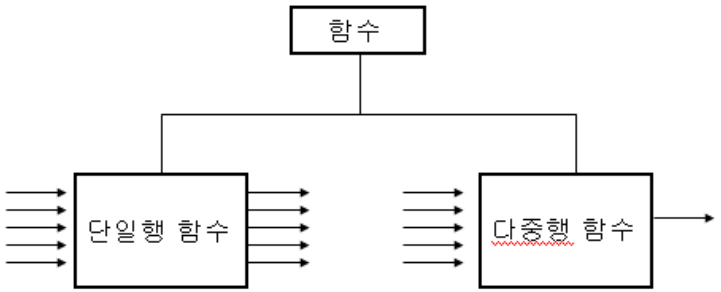
chapter 04.

단일 함수

▪ 제공되는 함수들은 기본적인 query문을 더욱 강력하게 해주고 데이터 값을 조작하는데 사용할 수 있다.

▪ SQL 함수의 특징 및 이점

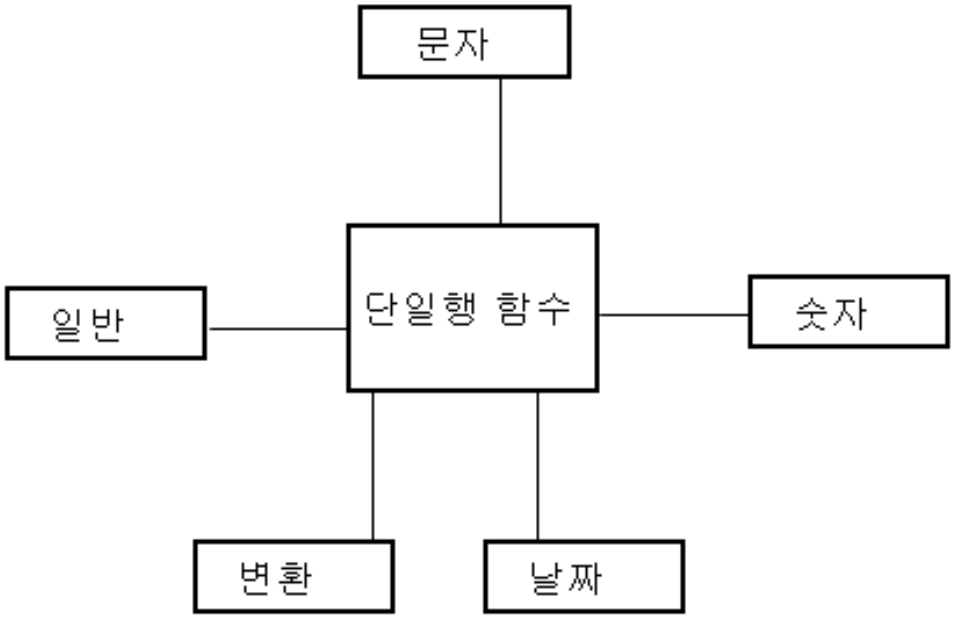
- 데이터에 계산을 수행할 수 있다.
- 개별적인 데이터 항목을 수정할 수 있다.
- 행의 그룹에 대해 결과를 조작할 수 있다.
- 출력을 위한 날짜와 숫자 형식을 조작할 수 있다.
- 열의 자료형을 변환할 수 있다.



▪ 함수의 종류
  ▪ 단일 행 함수
    - 모든 행에 대해서 각각 적용되어 행의 개수와 동일한 개수의 결과를 리턴.
    - SELECT, WHERE, ORDER BY절에 사용 가능하다.

  ▪ 다중 행 함수 (그룹 함수)
    - 검색되는 모든 행에 대해서 한번만 적용되고 한 건의 결과만을 리턴.

■ 단일 행 함수의 종류



https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/string-functions.html

14.8 String Functions and Operators

14.8.1 String Comparison Functions and Operators
14.8.2 Regular Expressions
14.8.3 Character Set and Collation of Function Results

Table 14.12 String Functions and Operators

Name	Description
ASCII()	Return numeric value of left-most character
BIN()	Return a string containing binary representation of a numbe
BIT_LENGTH()	Return length of argument in bits
CHAR()	Return the character for each integer passed
CHAR_LENGTH()	Return number of characters in argument
CHARACTER_LENGTH()	Synonym for CHAR_LENGTH()
CONCAT()	Return concatenated string
CONCAT_WS()	Return concatenate with separator
ELT()	Return string at index number
EXPORT_SET()	Return a string such that for every bit set in the value bits, yo and for every unset bit, you get an off string
FIELD()	Index (position) of first argument in subsequent arguments
FIND_IN_SET()	Index (position) of first argument within second argument
FORMAT()	Return a number formatted to specified number of decimal p
FROM_BASE64()	Decode base64 encoded string and return result
HEX()	Hexadecimal representation of decimal or string value
INSERT()	Insert substring at specified position up to specified number
INSTR()	Return the index of the first occurrence of substring
LCASE()	Synonym for LOWER()
LENGTH()	Return the length of a string in bytes
LIKE	Simple pattern matching
LOAD_FILE()	Load the named file
LOCATE()	Return the position of the first occurrence of substring
LOWER()	Return the argument in lowercase
LPAD()	Return the string argument, left-padded with the specified string
LTRIM()	Remove leading spaces
MAKE_SET()	Return a set of comma-separated strings that have the corresponding bit in bits set
REPEAT()	Repeat a string the specified number of times
REPLACE()	Replace occurrences of a specified string
REVERSE()	Reverse the characters in a string
RIGHT()	Return the specified rightmost number of characters
RLIKE	Whether string matches regular expression
RPAD()	Append string the specified number of times
RTRIM()	Remove trailing spaces
SOUNDEX()	Return a soundex string
SOUNDS LIKE	Compare sounds
SPACE()	Return a string of the specified number of spaces
STRCMP()	Compare two strings
SUBSTR()	Return the substring as specified
SUBSTRING()	Return the substring as specified

▪ LOWER 함수

– 대소문자가 혼합되어 있거나 대문자인 문자열을 소문자로 변경.

문법: LOWER(str)

```
SELECT empno, lower(ename),  LOWER('HeLLO')
FROM emp;
```

▪ UPPER 함수

– 대소문자가 혼합되어 있거나 소문자인 문자열을 대문자로 변경.

문법: UPPER(str)

```
SELECT empno, UPPER(ename),  UPPER('HeLLO')
FROM emp;
```

▪ CONCAT 함수

– 여러 개의 문자열을 연결한다.

문법: CONCAT(str1,str2,...)

```
SELECT empno, CONCAT(ename, ' ', sal)
FROM emp;
```

▪ CONCAT_WS 함수

– 구분자와 함께 여러 개의 문자열을 연결한다.

문법: CONCAT_WS(구분자,str1,str2,...)

```
SELECT empno, CONCAT_WS('/', ename, sal)
FROM emp;
```

▪ LPAD 함수

– 왼쪽에 문자열을 끼워 넣는 역할을 한다.
len은 반환되는 문자열의 전체 길이를 나타낸다.

문법: LPAD(str,len,padstr)

```
SELECT empno, LPAD(ename,10, '*'),  LPAD(sal, 10, '*')
FROM emp;
```

Result Grid | Filter Rows: | Export:

empno	LPAD(ename,10,'*')	LPAD(sal, 10, '*')
7369	*****SMITH	*******800
7499	*****ALLEN	******1600
7521	******WARD	******1250
7566	*****JONES	******2975
7654	****MARTIN	******1250
7698	*****BLAKE	******2850

▪ RPAD 함수

– LPAD와 반대로 오른쪽에 문자열을 끼워 넣는 역할을 한다.

문법: RPAD(str,len,padstr)

```
SELECT empno, RPAD(ename,10, '*'),  RPAD(sal, 10, '*')
FROM emp;
```

Result Grid | Filter Rows: | Export:

empno	RPAD(ename,10,'*')	RPAD(sal, 10, '*')
7369	SMITH*****	800*******
7499	ALLEN*****	1600******
7521	WARD******	1250******
7566	JONES*****	2975******
7654	MARTIN****	1250******
7698	BLAKE*****	2850******

- LEFT 함수
– 왼쪽에서 n글자 추출.

문법: LEFT(str,len)

```
SELECT LEFT('hello', 2);
```

Result Grid

	LEFT('hello', 2)
▶	he

- RIGHT 함수
– 오른쪽에서 n글자 추출.

문법: RIGHT(str,len)

```
SELECT RIGHT('hello', 2);
```

Result Grid

	RIGHT('hello', 2)
▶	lo

- SUBSTR 함수 (SUBSTR() is a synonym for SUBSTRING())

– pos 번째 자리부터 길이가 len개인 문자열을 반환 한다.
– pos는 순방향은 1부터 시작하고 역방향은 -1부터 시작한다.

문법: SUBSTR(str,pos [,len])

```
SELECT empno, ename, SUBSTR(ename,1,2), SUBSTR(ename,3),
       SUBSTR(ename, -1)
FROM emp;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

empno	ename	SUBSTR(ename,1,2)	SUBSTR(ename,3)	SUBSTR(ename, -1)
7369	SMITH	SM	ITH	H
7499	ALLEN	AL	LEN	N
7521	WARD	WA	RD	D
7566	JONES	JO	NES	S

- SUBSTR_INDX 함수

– 지정한 구분자가 나타나는 횟수를 기준으로 문자열을 반환

문법: SUBSTR_INDEX(str, delim, count)

```
SELECT SUBSTRING_INDEX('www.mysql.com', '.', 2);
```

Result Grid | Filter Rows:

	SUBSTRING_INDEX('www.mysql.com', '.', 2)
▶	www.mysql

- LENGTH 함수

– 문자열의 byte 수를 반환한다.
– 한글은 글자당 3byte 로 처리

문법: LENGTH(str)

```
SELECT empno, ename, LENGTH(ename)
FROM emp;
```

Result Grid | Filter Rows:

empno	ename	LENGTH(ename)
7369	SMITH	5
7499	ALLEN	5
7521	WARD	4
7566	JONES	5
7654	MARTIN	6
7698	BLAKE	5
7782	CLARK	5

```
SELECT empno, ename, LENGTH(ename), LENGTH('홍길동')
FROM emp;
```

- CHAR_LENGTH 함수

– 문자열의 길이를 반환한다.

문법: CHAR_LENGTH(str)

```
SELECT empno, ename, CHAR_LENGTH(ename)
FROM emp;
```

Result Grid | Filter Rows:

empno	ename	CHAR_LENGTH(ename)
7369	SMITH	5
7499	ALLEN	5
7521	WARD	4
7566	JONES	5
7654	MARTIN	6

```
SELECT LENGTH('홍길동'), CHAR_LENGTH('홍길동');
```

- REPLACE 함수

– 특정 문자열을 치환하는 함수 .

문법: REPLACE(str, from, to)

```
SELECT empno, ename, sal, REPLACE(sal, '0','o')
FROM emp;
```

Result Grid | Filter Rows:

empno	ename	sal	REPLACE(sal, '0','o')
7369	SMITH	800	8oo
7499	ALLEN	1600	16oo
7521	WARD	1250	125o
7566	JONES	2975	2975
7654	MARTIN	1250	125o
7698	BLAKE	2850	285o

- INSERT 함수

– 문자열의 특정 위치에서 지정한 길이만큼의 문자열을 삭제하고, 새로운 문자열로 교체

문법: INSERT(str, pos, len, newstr)

```
SELECT INSERT('Hello World', 7, 5, 'MySQL');
```

Result Grid | Filter Rows:

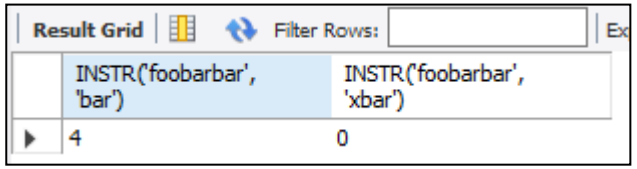
INSERT('Hello World', 7, 5, 'MySQL')
Hello MySQL

▪ INSTR 함수

– 문자열이 포함되어 있는지를 조사하여 문자열의 위치를 반환 한다.
– 지정한 문자열이 발견되지 않으면 0 값 반환 됩니다.

문법: INSTR(str, substr)

```
SELECT INSTR('foobarbar', 'bar'), INSTR('foobarbar', 'xbar');
```

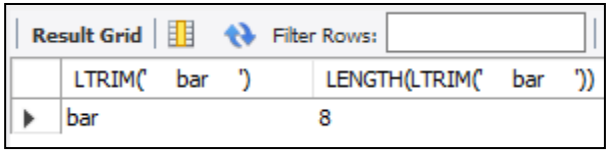

Result Grid | Filter Rows: | Ex

	INSTR('foobarbar', 'bar')	INSTR('foobarbar', 'xbar')
▶	4	0

▪ LTRIM함수
–문자열의 첫 문자부터 확인해서 공백 문자가 나타나는 동안 공백 문자를 제거.

문법: LTRIM(str)

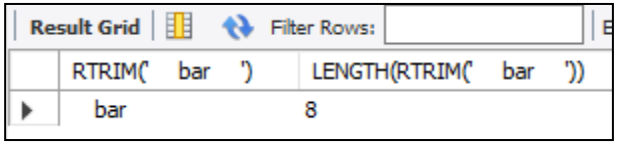
```
SELECT LTRIM('      bar      '), LENGTH(LTRIM('      bar      '));
```



▪ RTRIM함수
–문자열의 끝 문자부터 확인해서 공백 문자가 나타나는 동안 공백 문자를 제거.

문법: RTRIM(str)

```
SELECT RTRIM('      bar      '), LENGTH(RTRIM('      bar      '));
```



▪ TRIM함수

–문자열에서 머리말,꼬리말 또는 양쪽에 있는 공백 문자 및 특정문자를 제거.
(leading | trailing | both 가 생략되면 both가 기본)

문법: TRIM([BOTH|LEADING|TRAILING from] str)

```
SELECT TRIM('      bar      '),
       TRIM(BOTH FROM '      bar      '),
       TRIM(LEADING FROM '      bar      '),
       TRIM(TRAILING FROM '      bar      ');
```

```
SELECT TRIM('xxxbarxxx'),
       TRIM(BOTH 'x' FROM 'xxxbarxxx'),
       TRIM(LEADING 'x' FROM 'xxxbarxxx'),
       TRIM(TRAILING 'x' FROM 'xxxbarxxx');
```

▪ REVERSE함수

–문자열 값을 거꾸로 반환한다.

문법: REVERSE(str)

```
SELECT ename, REVERSE(ename)
FROM emp;
```

Result Grid | Filter Rows:

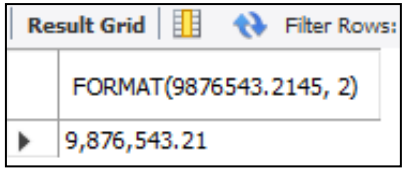
ename	REVERSE(ename)
SMITH	HTIMS
ALLEN	NELLA
WARD	DRAW
JONES	SENOJ
MARTIN	NITRAM
BLAKE	EKALB

▪ FORMAT함수

–지정된 숫자값을 '#,###,###.##' 같은 포맷을 적용하여 문자열로 반환.

문법: FORMAT(정수, 소수점자릿수[,locale])

```
SELECT FORMAT(9876543.2145, 2);
```



▪ 정규표현식 (Regex)

-문자열이 특정 규칙에 맞는지 검색하거나 검사하는 표현식

기호	의미	예제
.	임의의 문자 1개	a.c ex: abc, a1c
^	문자열의 시작	^김 ex: 김유신, 김홍도
$	문자열의 끝	com$ ex: google.com , github.com
*	앞 문자가 0 개 이상	ab* ex: a, ab, abb
+	앞 문자가 1 개 이상	ab+ ex: ab, abb
?	앞 문자가 0 개 또는 1개	ab? ex: a, ab
[ ]	지정한 문자 중 하나	[abc] ex: a, b , c
{n}	앞 문자가 정확히 n개	[0-9]{3} ex: 0~9 사이의 숫자 3개

▪ 자주 사용하는 문자 패턴

표현식	의미
[0-9]	숫자 1개
[a-z]	영문 소문자 1개
[A-Z]	영문 대문자 1개
[가-힣]	한글 1개
[0-9]+	숫자 1개 이상
[a-zA-Z]+	영문 1개 이상
[^0-9]	숫자가 아닌 문자 1개

▪ REGEXP_LIKE 함수

– 정규표현식과 일치여부 반환 (1: 일치 0:불일치)

문법: REGEXP_LIKE(str, pat)

```
SELECT ename, REGEXP_LIKE(ename, '^M')
FROM emp;
```

Result Grid | Filter Rows:

ename	REGEXP_LIKE(ename, '^M')
SMITH	0
ALLEN	0
WARD	0
JONES	0
MARTIN	1
BLAKE	0
CLARK	0

▪ REGEXP_REPLACE 함수

– 정규표현식과 일치하는 부분 교체

문법: REGEXP_REPLACE(str, pat, repl [, pos [, occurrence]])

```
SELECT REGEXP_REPLACE('a1b2', '[0-9]', '');
```

Result Grid | Filter Rows:

	REGEXP_REPLACE('a1b2', '[0-9]', '')
▶	ab

```
SELECT REGEXP_REPLACE('a1b2', '[0-9]', '' ,1 ,2);
```

Result Grid | Filter Rows:

	REGEXP_REPLACE('a1b2', '[0-9]', '', 1, 2)
▶	a1b

▪ REGEXP_SUBSTR 함수

– 정규표현식과 일치하는 문자열을 찾아서 반환

문법: REGEXP_SUBSTR(str, pat [, pos [, occurrence]])

```
SELECT REGEXP_SUBSTR('a1987b2c3', '[0-9]');                 -- 1
SELECT REGEXP_SUBSTR('a1987b2765c3', '[0-9]+');             -- 1987
SELECT REGEXP_SUBSTR('a1987b2765c3', '[0-9]+', 1, 2);       -- 2765
SELECT REGEXP_SUBSTR('123Hello456', '[A-Za-z]+');           -- Hello
```

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/numeric-functions.html

14.6 Numeric Functions and Operators

14.6.1 Arithmetic Operators
14.6.2 Mathematical Functions

Table 14.8 Numeric Functions and Operators

Name	Description
%, MOD	Modulo operator
*	Multiplication operator
+	Addition operator
-	Minus operator
-	Change the sign of the argument
/	Division operator
ABS()	Return the absolute value
ACOS()	Return the arc cosine
ASIN()	Return the arc sine
ATAN()	Return the arc tangent
ATAN2(), ATAN()	Return the arc tangent of the two arguments
CEIL()	Return the smallest integer value not less than the argument
CEILING()	Return the smallest integer value not less than the argument
CONV()	Convert numbers between different number bases
COS()	Return the cosine
COT()	Return the cotangent
CRC32()	Compute a cyclic redundancy check value
DEGREES()	Convert radians to degrees
DIV	Integer division
EXP()	Raise to the power of
FLOOR()	Return the largest integer value not greater than the argument
LN()	Return the natural logarithm of the argument
LOG()	Return the natural logarithm of the first argument
LOG10()	Return the base-10 logarithm of the argument
LOG2()	Return the base-2 logarithm of the argument
MOD()	Return the remainder
PI()	Return the value of pi
POW()	Return the argument raised to the specified power
POWER()	Return the argument raised to the specified power
RADIANS()	Return argument converted to radians
RAND()	Return a random floating-point value
ROUND()	Round the argument
SIGN()	Return the sign of the argument
SIN()	Return the sine of the argument
SQRT()	Return the square root of the argument
TAN()	Return the tangent of the argument
TRUNCATE()	Truncate to specified number of decimal places

- ROUND 함수 (반올림)

–숫자를 소수점 n자리까지 반올림해서 출력한다. 생략하면 기본값은 0.
–n이 양수이면 소수 자리를, 음수이면 정수 자리를 반올림한다.

문법: ROUND(정수 [, n])

```
SELECT ROUND(45.678), ROUND(45.678, 2) , ROUND(45.678, -1);
```

Result Grid | Filter Rows: | Export:

ROUND(45.678)	ROUND(45.678, 2)	ROUND(45.678, -1)
46	45.68	50

- TRUNCATE 함수 (버림)

–숫자를 소수 n 자리까지 절삭해서 출력한다. 생략불가
– n이 양수이면 소수 자리를, 음수이면 정수 자리를 절삭.

문법: TRUNCATE(정수 , n)

```
SELECT TRUNCATE(45.678, 0), TRUNCATE(45.678, 2) ,
       TRUNCATE(45.678, -1);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

	TRUNCATE(45.678, 0)	TRUNCATE(45.678, 2)	TRUNCATE(45.678, -1)
▶	45	45.67	40

▪ CEIL(CEILING) 함수 (올림)

– 주어진 숫자보다 크거나 같은 최소 정수 리턴.

문법: CEIL(실수값)

```
SELECT CEIL(45.178), CEIL(-45.178);
```

Result Grid | Filter Rows:

	CEIL(45.178)	CEIL(-45.178)
▶	46	-45

▪ FLOOR 함수 (내림)

– 주어진 숫자보다 작거나 같은 최대 정수 리턴.

문법: FLOOR(실수값)

```
SELECT FLOOR(45.178), FLOOR(-45.178);
```

Result Grid | Filter Rows:

	FLOOR(45.178)	FLOOR(-45.178)
▶	45	-46

- MOD 함수

– 숫자의 나머지를 구하는 함수로서 % 연산자와 동일
– n으로 나누어 남은 값을 반환한다. n이 0일 경우 그 자체를 반환.

문법: MOD(m, n)

```
SELECT MOD(10, 3), 10%3 ;
```

- SIGN 함수

– 숫자의 부호를 식별하는 함수로서 음수(-1), 양수(1), 0(0) 값 반환

문법: SIGN(값)

```
SELECT SIGN(-1.200), SIGN(34.3), SIGN(0);
```

Result Grid | Filter Rows:

	SIGN(-1.200)	SIGN(34.3)	SIGN(0)
▶	-1	1	0

- RAND 함수

– 0 이상 1 미만의 난수 반환

문법: RAND()

```
SELECT RAND();                          --  0 <= R < 1
SELECT FLOOR(7 + (RAND() * 5));         --  7 <= R < 12
SELECT FLOOR(1 + RAND() * 10);          --  1 <= R < 11
```

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/date-and-time-functions.html

14.7 Date and Time Functions

This section describes the functions that can be used to manipulate temporal values. See Section 13.2, "Date and Time Data Types", for a description of the range of values each date and time type has and the valid formats in which values may be specified.

Table 14.11 Date and Time Functions

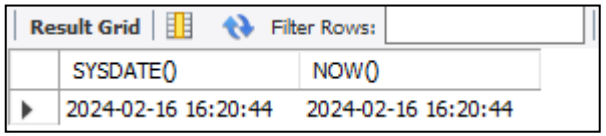
Name	Description
ADDDATE()	Add time values (intervals) to a date value
ADDTIME()	Add time
CONVERT_TZ()	Convert from one time zone to another
CURDATE()	Return the current date
CURRENT_DATE(), CURRENT_DATE	Synonyms for CURDATE()
CURRENT_TIME(), CURRENT_TIME	Synonyms for CURTIME()
CURRENT_TIMESTAMP(), CURRENT_TIMESTAMP	Synonyms for NOW()
CURTIME()	Return the current time
DATE()	Extract the date part of a date or datetime expression
DATE_ADD()	Add time values (intervals) to a date value
DATE_FORMAT()	Format date as specified
DATE_SUB()	Subtract a time value (interval) from a date
DATEDIFF()	Subtract two dates
MONTHNAME()	Return the name of the month
NOW()	Return the current date and time
PERIOD_ADD()	Add a period to a year-month
PERIOD_DIFF()	Return the number of months between periods
QUARTER()	Return the quarter from a date argument
SEC_TO_TIME()	Converts seconds to 'hh:mm:ss' format
SECOND()	Return the second (0-59)
STR_TO_DATE()	Convert a string to a date
SUBDATE()	Synonym for DATE_SUB() when invoked with three arguments
SUBTIME()	Subtract times
SYSDATE()	Return the time at which the function executes
TIME()	Extract the time portion of the expression passed
TIME_FORMAT()	Format as time
TIME_TO_SEC()	Return the argument converted to seconds
TIMEDIFF()	Subtract time
TIMESTAMP()	With a single argument, this function returns the date or datetime expression; with two arguments, the sum of the arguments

- SYSDATE 및 NOW 함수

– 데이터베이스 서버에 설정되어 있는 날짜 및 시간을 리턴.
출력 포맷은 ' YYYY-MM-DD hh:mm:ss ' 이다.

문법: SYSDATE() 및 NOW()

```
SELECT SYSDATE(), NOW();
```

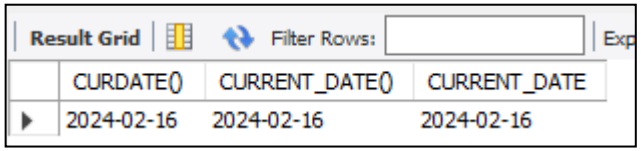


- CURDATE 함수

– 데이터베이스 서버에 설정되어 있는 날짜를 리턴.
출력 포맷은 'YYYY-MM-DD' 형식의 문자열 포맷이다.
– current_date() 와 current_date 는 curdate()의 synonym이다.

문법: CURDATE()

```
SELECT CURDATE(), CURRENT_DATE(), CURRENT_DATE;
```


Result Grid | Filter Rows: | Exp

	CURDATE()	CURRENT_DATE()	CURRENT_DATE
▶	2024-02-16	2024-02-16	2024-02-16

- CURTIME 함수

– 데이터베이스 서버에 설정되어 있는 시간을 리턴.
출력 포맷은 'hh:mm:ss' 형식의 문자열 포맷이다.
– current_time() 와 current_time 는 curtime()의 synonym이다.

문법: CURTIME()

```
SELECT CURTIME(),  CURRENT_TIME(),  CURRENT_TIME;
```

Result Grid | Filter Rows:

	CURTIME()	CURRENT_TIME()	CURRENT_TIME
▶	16:09:30	16:09:30	16:09:30

- 날짜 및 시간 추출 함수

– 날짜에서 년/월/일 추출가능
– 시간에서 시/분/초 추출가능

함수	기능	예제
YEAR()	연도	YEAR('2026-10-11') 결과: 2026
MONTH()	월	MONTH('2026-10-11') 결과: 10
DAY()	일	DAY('2026-10-11') 결과: 11
HOUR()	시	HOUR('14:35:20') 결과: 14
MINUTE()	분	MINUTE('14:35:20') 결과: 35
SECOND()	초	SECOND('14:35:20') 결과: 20

```
SELECT YEAR('2026-10-11');
SELECT MONTH('2026-10-11');
SELECT DAY('2026-10-11');
SELECT HOUR('14:35:20');
SELECT MINUTE('14:35:20');
SELECT SECOND('14:35:20');
```

■ 날짜 함수

– 날짜 함수는 mysql 날짜에 대해 연산을 한다.

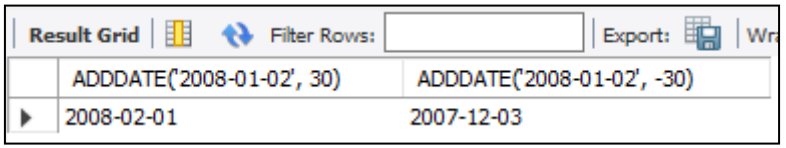
함수명	설명
ADDDATE	날짜에 일(day) 더하기 및 빼기
DATE_ADD	날짜에 입력된 기간(interval)만큼 더한다.
DATE_SUB	날짜에 입력된 기간(interval)만큼 뺀다
DATEDIFF	날짜간 일(day) 차이를 반환한다
TIMESTAMPDIFF	날짜에 지정된 단위(unit)만큼 차이를 반환한다.
LAST_DAY	월의 마지막 날짜를 반환한다.
EXTRACT	날짜에서 지정된 단위(unit)에 해당하는 날짜 정보 추출
DATE_FORMAT	지정된 날짜를 format 적용하여 문자열로 반환한다.
STR_TO_DATE	지정된 문자를 format 적용하여 날짜로 반환한다.

- ADDDATE 함수

-날짜에 일(day)을 더하거나 뺀다.

문법: ADDDATE(date, days)

```
SELECT ADDDATE('2008-01-02', 30), ADDDATE('2008-01-02', -30);
```



- DATE_ADD 함수

–날짜에 입력된 기간(interval)만큼 더하거나 뺀다.

문법: DATE_ADD(date, INTERVAL expr unit)

```
SELECT DATE_ADD('2008-01-02', INTERVAL 1 DAY) as A1,
       DATE_ADD('2008-01-02', INTERVAL 1 MONTH) as A2,
       DATE_ADD('2008-01-02', INTERVAL 1 YEAR) as A3,
       NOW() as A4,
       DATE_ADD(NOW(), INTERVAL 10 MINUTE) as A5,
       DATE_ADD(NOW(), INTERVAL 2 HOUR) as A6;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

A1	A2	A3	A4	A5	A6
2008-01-03	2008-02-02	2009-01-02	2024-02-16 17:18:36	2024-02-16 17:28:36	2024-02-16 19:18:36

- DATE_SUB 함수

–날짜에 입력된 기간(interval)만큼 빼거나 더한다.

문법: DATE_SUB(date, INTERVAL expr unit)

```
SELECT DATE_SUB('2008-01-02', INTERVAL 1 DAY) as B1,
       DATE_SUB('2008-01-02', INTERVAL 1 MONTH) as B2,
       DATE_SUB('2008-01-02', INTERVAL 1 YEAR) as B3,
       NOW() as B4,
       DATE_SUB(NOW(), INTERVAL 10 MINUTE) as B5,
       DATE_SUB(NOW(), INTERVAL 2 HOUR) as B6;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

B1	B2	B3	B4	B5	B6
2008-01-01	2007-12-02	2007-01-02	2024-02-16 17:22:39	2024-02-16 17:12:39	2024-02-16 15:22:39

- DATEDIFF함수

–날짜간 일(day) 차이를 반환한다.

문법: DATEDIFF(date1, date2)

```
SELECT DATEDIFF('2023-01-04', '2022-01-04');
```

Result Grid | Filter Rows:

	DATEDIFF('2023-01-04', '2022-01-04')
▶	365

- TIMESTAMPDIFF함수

–날짜에 지정된 단위(unit)만큼 차이를 반환한다.

문법: TIMESTAMPDIFF(unit, date1, date2)

```
SELECT TIMESTAMPDIFF(HOUR, '2020-3-1', '2020-3-3') AS C1,
       TIMESTAMPDIFF(DAY, '2020-3-1', '2020-3-3') AS C2,
       TIMESTAMPDIFF(MONTH, '2020-2-1', '2020-6-3') AS C3,
       TIMESTAMPDIFF(YEAR, '2020-3-1', '2022-3-3') AS C4;
```

Result Grid | Filter Rows:

	C1	C2	C3	C4
▶	48	2	4	2

- LAST_DAY 함수

– 월의 마지막 날짜를 계산
– 윤년, 평년은 자동 계산

문법: LAST_DAY(date)

```
SELECT LAST_DAY('2003-02-05'), LAST_DAY(NOW());
```

Result Grid | Filter Rows:

LAST_DAY('2003-02-05')	LAST_DAY(NOW())
2003-02-28	2024-02-29

▪ EXTRACT 함수

– 날짜에서 지정된 단위(unit)에 해당하는 날짜 정보 추출.

문법: EXTRACT(unit FROM date)

```
SELECT NOW(),
       EXTRACT(SECOND FROM NOW()),
       EXTRACT(MINUTE FROM NOW()),
       EXTRACT(HOUR FROM NOW()),
       EXTRACT(DAY FROM NOW()),
       EXTRACT(MONTH FROM NOW()),
       EXTRACT(YEAR FROM NOW()),
       EXTRACT(YEAR_MONTH FROM NOW());
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

NOW()	EXTRACT(SECOND FROM NOW())	EXTRACT(MINUTE FROM NOW())	EXTRACT(HOUR FROM NOW())	EXTRACT(DAY FROM NOW())	EXTRACT(MONTH FROM NOW())	EXTRACT(YEAR FROM NOW())	EXTRACT(YEAR_MONTH FROM NOW())
2024-02-16 17:34:28	28	34	17	16	2	2024	202402

▪ DATE_FORMAT 함수

– 지정된 날짜를 format 적용하여 문자열로 반환.

문법: DATE_FORMAT(date, format)

Specifier	Description
%a	Abbreviated weekday name (Sun..Sat)
%b	Abbreviated month name (Jan..Dec)
%c	Month, numeric (0..12)
%D	Day of the month with English suffix (0th
%d	Day of the month, numeric (00..31)
%e	Day of the month, numeric (0..31)
%f	Microseconds (000000..999999)
%H	Hour (00..23)
%h	Hour (01..12)
%I	Hour (01..12)
%i	Minutes, numeric (00..59)
%j	Day of year (001..366)
%k	Hour (0..23)

%l	Hour (1..12)
%M	Month name (January..December)
%m	Month, numeric (00..12)
%p	AM or PM
%r	Time, 12-hour (hh:mm:ss followed by AM or PM)
%S	Seconds (00..59)
%s	Seconds (00..59)
%T	Time, 24-hour (hh:mm:ss)

```
SELECT NOW(),
       DATE_FORMAT(NOW(),'%Y%m%d'),
       DATE_FORMAT(NOW(),'%Y/%m/%d'),
       DATE_FORMAT(NOW(),'%Y년%m월 %d일'),
       DATE_FORMAT(NOW(),'%H:%i:%S'),
       DATE_FORMAT(NOW(),'%Y');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

NOW()	DATE_FORMAT(NOW(),'%Y%m%d')	DATE_FORMAT(NOW(),'%Y/%m/%d')	DATE_FORMAT(NOW(),'%Y년%m월 %d일')	DATE_FORMAT(NOW(),'%H:%i:%S')	DATE_FORMAT(NOW(),'%Y')
2024-02-16 17:39:01	20240216	2024/02/16	2024년02월 16일	17:39:01	2024

▪ STR_TO_DATE 함수

– 지정된 문자열을 format 적용하여 날짜로 반환.

문법: STR_TO_DATE(date, format)

```
SELECT STR_TO_DATE('2020-03-04','%Y-%m-%d'),
       STR_TO_DATE('01,5,2013','%d,%m,%Y'),
       STR_TO_DATE('2020년03월05일','%Y년%m월%d일'),
       STR_TO_DATE('2020-03-04 12:23:32','%Y-%m-%d %H:%i:%S');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

STR_TO_DATE('2020-03-04','%Y-%m-%d')	STR_TO_DATE('01,5,2013','%d,%m,%Y')	STR_TO_DATE('2020년03월05일','%Y년%m월%d일')	STR_TO_DATE('2020-03-04 12:23:32','%Y-%m-%d %H:%i:%S')
2020-03-04	2013-05-01	2020-03-05	2020-03-04 12:23:32

▪ CASE 함수

문법 1: 동등비교

```
CASE value WHEN  value1 THEN result
           [WHEN  value2 THEN result ...]
           [ELSE result] END
```

```
SELECT empno, ename, sal, job,
       CASE job WHEN 'ANALYST' THEN sal * 1.1
                WHEN 'CLERK' THEN sal * 1.2
                WHEN 'MANAGER' THEN sal * 1.3
                WHEN 'PRESIDENT' THEN sal * 1.4
                WHEN 'SALESMAN' THEN sal * 1.5
                ELSE sal
       END AS 실수령
FROM emp;
```

Result Grid | Filter Rows: | Export:

empno	ename	sal	job	실수령
7369	SMITH	800	CLERK	960
7499	ALLEN	1600	SALESMAN	2400
7521	WARD	1250	SALESMAN	1875
7566	JONES	2975	MANAGER	3867.5
7654	MARTIN	1250	SALESMAN	1875
7698	BLAKE	2850	MANAGER	3705
7782	CLARK	2450	MANAGER	3185

▪ CASE 함수

문법2: 부등비교

```
CASE  WHEN condition THEN result
      [WHEN condition THEN result ...]
      [ELSE result] END
```

```
SELECT empno, ename, sal,
       CASE      WHEN sal > 3500 THEN '이사급'
                 WHEN sal > 2500 THEN '과장급'
                 ELSE '사원급'
       END AS 직급연봉
FROM emp;
```

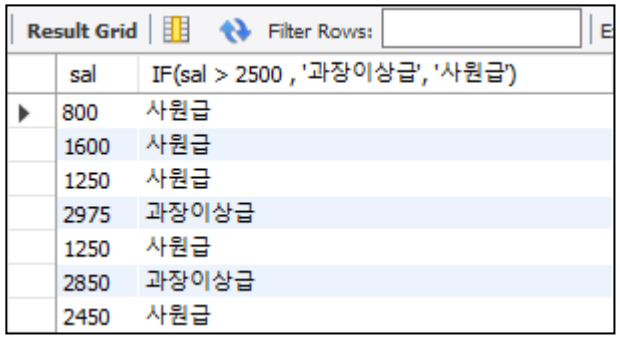
Result Grid | Filter Rows:

	empno	ename	sal	직급연봉
▶	7369	SMITH	800	사원급
	7499	ALLEN	1600	사원급
	7521	WARD	1250	사원급
	7566	JONES	2975	과장급
	7654	MARTIN	1250	사원급
	7698	BLAKE	2850	과장급
	7782	CLARK	2450	사원급

▪ IF 함수

문법: IF (조건식, 참, 거짓)

```
SELECT sal, IF(sal > 2500 , '과장이상급', '사원급')
FROM emp;
```



Result Grid | Filter Rows:

sal	IF(sal > 2500 , '과장이상급', '사원급')
800	사원급
1600	사원급
1250	사원급
2975	과장이상급
1250	사원급
2850	과장이상급
2450	사원급

▪ CAST 함수

문법: CAST(값 as 변환할 타입)

```
select '10',
       cast('123' as SIGNED ),
       cast(100 as char ),
       cast('12.34' as float ),
       cast('24-03-29' as DATE ),
       cast('240329' as DATE ),
       cast('24-03-29 03:23:34' as DATETIME );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	10	cast('123' as SIGNED)	cast(100 as char)	cast('12.34' as float)	cast('24-03-29' as DATE)	cast('240329' as DATE)	cast('24-03-29 03:23:34' as DATETIME)
▶	10	123	100	12.34	2024-03-29	2024-03-29	2024-03-29 03:23:34

chapter 05.

그룹 함수

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/aggregate-functions.html

14.19.1 Aggregate Function Descriptions

This section describes aggregate functions that operate on sets of values. They are often used with a `GROUP BY` clause to group values into subsets.

Table 14.29 Aggregate Functions

Name	Description
`AVG()`	Return the average value of the argument
`BIT_AND()`	Return bitwise AND
`BIT_OR()`	Return bitwise OR
`BIT_XOR()`	Return bitwise XOR
`COUNT()`	Return a count of the number of rows returned
`COUNT(DISTINCT)`	Return the count of a number of different values
`GROUP_CONCAT()`	Return a concatenated string
`JSON_ARRAYAGG()`	Return result set as a single JSON array
`JSON_OBJECTAGG()`	Return result set as a single JSON object
`MAX()`	Return the maximum value
`MIN()`	Return the minimum value
`STD()`	Return the population standard deviation
`STDDEV()`	Return the population standard deviation
`STDDEV_POP()`	Return the population standard deviation
`STDDEV_SAMP()`	Return the sample standard deviation
`SUM()`	Return the sum
`VAR_POP()`	Return the population standard variance
`VAR_SAMP()`	Return the sample variance
`VARIANCE()`	Return the population standard variance

❑ 그룹 함수

- 단일 행 함수와는 달리 그룹 함수는 여러 행 또는 테이블 전체에 대해 함수가 적용되어 하나의 결과를 가져오는 함수를 의미한다.
- NULL은 대부분의 그룸 함수에서 제외된다.
- 그룹 당 하나의 결과가 주어지도록 명시적으로 GROUP BY절을 이용하여 그룹화 할 수 있고 HAVING 절을 이용하여 그룹에 대한 조건을 제한 할 수 있다.

▪ 그룹함수 종류

함수명	설명
SUM([distinct] exp)	총합
AVG([distinct] exp)	평균
MIN([distinct] exp) MAX([distinct] exp)	최소값, 최대값
COUNT([distinct] exp|*)	행 갯수
GROUP_CONCAT([distinct] exp [order by exp] [separator sep])	그룹 내 문자열을 하나로 연결

count(*) : null 값을 포함한 전체행의 수
count(표현식): null 값을 제외한 행의 수

EMP 테이블에서 모든 SALESMAN에 대한 급여의 평균, 최고액, 최저액, 합계를 출력.

```
SELECT AVG(sal), MAX(sal), MIN(sal), SUM(sal)
FROM emp
WHERE job = 'SALESMAN';
```

Result Grid | Filter Rows:

AVG(sal)	MAX(sal)	MIN(sal)	SUM(sal)
1400	1600	1250	5600

EMP 테이블에 등록되어 있는 인원수, 보너스에 null이 아닌 인원수, 보너스의 평균, 등록되어 있는 부서의 수를 구하여 출력.

```
SELECT COUNT(*), COUNT(comm),
       AVG(comm), AVG(IFNULL(comm,0)),
       COUNT(deptno), COUNT(DISTINCT deptno)
FROM emp;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

COUNT(*)	COUNT(comm)	AVG(comm)	AVG(IFNULL(comm,0))	COUNT(deptno)	COUNT(DISTINCT deptno)
14	4	550.0000	157.1429	14	3

EMP 테이블에서 부서별 사원 이름을 한 줄로 출력.

```
SELECT deptno, group_concat(ename)
FROM emp
group by deptno;

SELECT deptno, group_concat(ename ORDER BY ename DESC)
FROM emp
group by deptno;

SELECT deptno, group_concat(ename ORDER BY ename DESC SEPARATOR '-')
FROM emp
group by deptno;
```

deptno	group_concat(ename ORDER BY ename DESC SEPARATOR '-')
10	MILLER-KING-CLARK
20	SMITH-SCOTT-JONES-FORD-ADAMS
30	WARD-TURNER-MARTIN-JAMES-BLAKE-ALLEN

- GROUP BY
  - 전체 테이블이 아닌 특정 그룹으로 묶을 때 사용. (예: 부서별, 직급별)

```
SELECT [ column ,] 그룹함수( column), ...
[FROM    table]
[WHERE 조건식]
[GROUP BY column|별칭|컬럼순서]
[ORDER BY column|별칭|컬럼순서];
```

GROUP BY 작성 지침

- GROUP BY 없이 사용하면 전체 데이터를 하나의 그룹으로 계산한다.
- SELECT 절 뒤에 사용할 수 있는 컬럼은 GROUP BY 뒤에 기술된 컬럼 이어야 한다.
  (상수인 리터럴은 제외)
- WHERE 절을 사용하여 행을 그룹으로 분류하기 전에 제외 시킬 수 있다.
- HAVING절을 이용하여 GROUP BY 소그룹을 제외 시킬 수 있다.
- ORDER By와 동일하게 GROUP BY 절 뒤에도 컬럼 별칭 및 컬럼순서 위치값을 사용할 수 있다.
- WHERE 절에 그룹함수를 사용 할 수 없다.
  (그룹함수를 사용할 수 있는 GROUP By절보다 WHERE절이 먼저 수행된다.)

샘플) EMP 테이블에서 부서별로 인원수, 평균 급여, 최저급여, 최고 급여, 급여의 합을 구하여 출력?

```
SELECT deptno, COUNT(*), ROUND(AVG(sal)), MIN(sal), MAX(sal), SUM(sal)
FROM emp
GROUP BY deptno;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

deptno	COUNT(*)	ROUND(AVG(sal))	MIN(sal)	MAX(sal)	SUM(sal)
10	3	2917	1300	5000	8750
20	5	2175	800	3000	10875
30	6	1567	950	2850	9400

샘플) 각 부서별로 인원수, 급여의 평균, 최저 급여, 최고 급여, 급여의 합을 구하여 급여의 합이 많은 순으로 출력?

```
SELECT deptno, COUNT(*), ROUND(AVG(sal)), MIN(sal), MAX(sal), SUM(sal)
FROM emp
GROUP BY deptno
ORDER BY SUM(sal) DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell C

deptno	COUNT(*)	ROUND(AVG(sal))	MIN(sal)	MAX(sal)	SUM(sal)
20	5	2175	800	3000	10875
30	6	1567	950	2850	9400
10	3	2917	1300	5000	8750

샘플) 부서별, 업무별 그룹화하여 결과를 부서번호, 업무, 인원수, 급여의 평균, 급여의 합을 구하여 출력?

```
SELECT deptno, job, COUNT(*), ROUND(AVG(sal)), SUM(sal)
FROM emp
GROUP BY deptno,job
ORDER BY 1;
```

Result Grid | Filter Rows: | Export:

deptno	job	COUNT(*)	ROUND(AVG(sal))	SUM(sal)
10	CLERK	1	1300	1300
10	MANAGER	1	2450	2450
10	PRESIDENT	1	5000	5000
20	ANALYST	2	3000	6000
20	CLERK	2	950	1900
20	MANAGER	1	2975	2975
30	CLERK	1	950	950
30	MANAGER	1	2850	2850
30	SALESMAN	4	1400	5600

❑ Having

- HAVING 절
  : GROUP BY에 의해 분류된 그룹들을 제한하기 위한 방법.

5. SELECT [column ,] group_function(column), ...
1. FROM table
2. [WHERE 조건식]
3. [GROUP BY column]
4. [HAVING group_조건식
6. [ORDER BY column];

```
SELECT deptno, SUM(sal)
FROM emp
GROUP BY deptno
HAVING SUM(sal) > 7000;
```

Result Grid | Filter R

deptno	SUM(sal)
10	8750
20	10875
30	9400

```
SELECT deptno, SUM(sal)
FROM emp
WHERE sal > 1500
GROUP BY deptno
HAVING SUM(sal) > 7000;
```

Result Grid | Filt

deptno	SUM(sal)
10	7450
20	8975

샘플) EMP 테이블에서 급여가 최대 2900 이상인 부서에 대해서 부서번호, 평균 급여, 급여의 합을 구하여 출력?

```
SELECT deptno, ROUND(AVG(sal)), SUM(sal)
FROM emp
GROUP BY deptno
HAVING MAX(sal) > 2900;
```

샘플) EMP 테이블에서 업무별 급여의 평균이 2000 이상인 업무에 대해서 업무명, 평균 급여, 급여의 합을 구하여 출력?

```
SELECT job, ROUND(AVG(sal)), SUM(sal)
FROM emp
GROUP BY job
HAVING AVG(sal) > 2000;
```

샘플) EMP 테이블에서 전체 월급이 5000을 초과하는 각 업무에 대해서 업무와 월급 합계를 출력하여라.
단 판매원은 제외하고 월 급여 합계로 내림차순 정렬한다.

```
SELECT job, SUM(sal)
FROM emp
WHERE job != 'SALESMAN'
GROUP BY job
HAVING SUM(sal) > 5000
ORDER BY SUM(sal) DESC;
```

chapter 06.

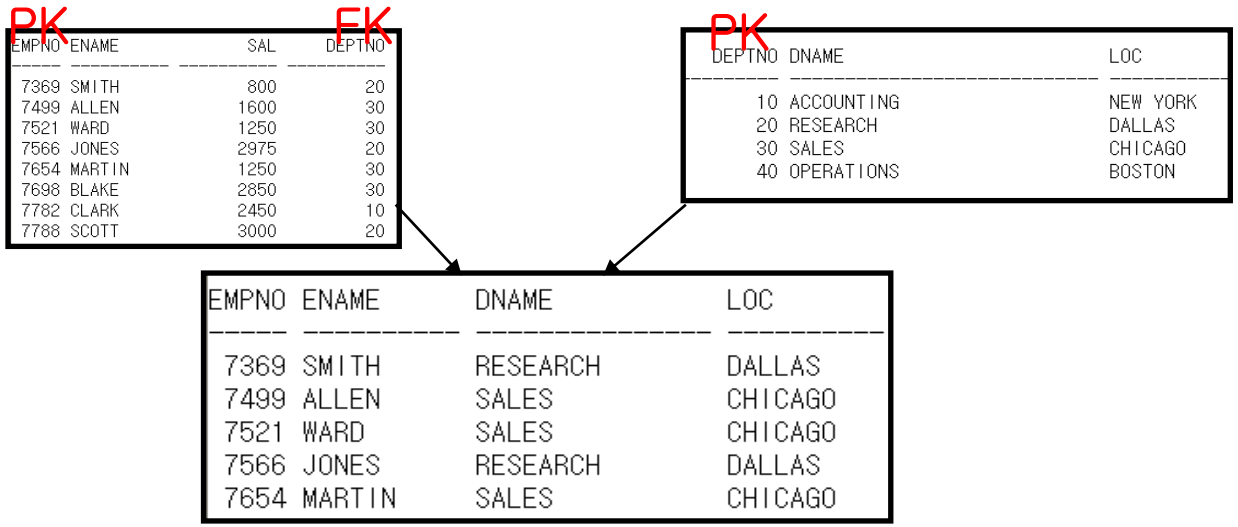
조인

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/join.html

- JOIN 정의

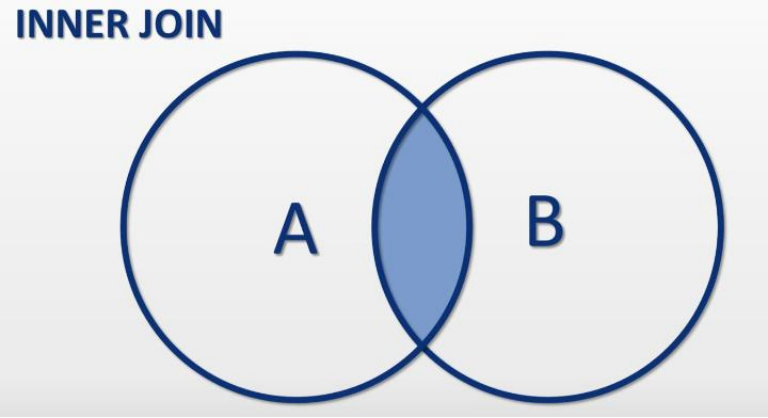
  - 검색하고자 하는 컬럼이 한 개의 테이블이 아닌 여러 개의 테이블에 존재하는 경우에 사용되는 기술.

  - 일반적으로 Primary Key(PK 로 사용)와 Foreign Key(FK 로 사용)을 사용하여 Join 하는 경우가 대부분이지만 때로는 논리적인 값들의 연관으로 Join 하는 경우도 있다.



- JOIN 종류 1

  Inner 조인
  - 양쪽 테이블에서 조인 조건이 일치하는 행만 조회.



Outer 조인
- 양쪽 테이블에서 조인 조건이 일치하지 않는 행까지 포함해서 조회





- JOIN 종류2
  - Cross 조인
  - Natural 조인
  - Using절 이용한 조인
  - On 절 이용한 조인
  - Outer 조인 (left|right)
  - self 조인

- ANSI 조인 특징

  - Join 조건은 FROM 절에서 지정된다.
  - 검색 조건은 WHERE 절에서 지정된다.
  - table에 별칭 지정 가능, 별칭 지정후에는 table명 사용불가
  - USING절에서는 공통 컬럼명에 별칭 사용 불가.

ANSI 조인 기본 문법

```
SELECT table1.column, [table2.]column, [별칭.]column
FROM table1 [별칭]
[CROSS JOIN table2 [별칭]] |
[NATURAL JOIN table2] |
[JOIN table2 USING (column)] |
[JOIN table2 ON (table1.column = table2.column)] |
[LEFT OUTER JOIN table2 ON (table1.column = table2.column)] |
[RIGHT OUTER JOIN table2 ON (table1.column = table2.column)]
```

❑ ANSI 조인 – Cross Join

- Cross Join
  - 두 개의 테이블의 레코드를 각각 cross해서 조인한다. (조합)
  - 실제 데이터로서 사용은 불가

```
SELECT empno, ename, dname
FROM emp CROSS JOIN dept;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

empno	ename	dname
7369	SMITH	OPERATIONS
7369	SMITH	SALES
7369	SMITH	RESEARCH
7369	SMITH	ACCOUNTING
7499	ALLEN	OPERATIONS
7499	ALLEN	SALES
7499	ALLEN	RESEARCH
7499	ALLEN	ACCOUNTING
7521	WARD	OPERATIONS
7521	WARD	SALES
7521	WARD	RESEARCH
7521	WARD	ACCOUNTING
7566	JONES	OPERATIONS
7566	JONES	SALES
7566	JONES	RESEARCH

Result 3

Output

Action Output

#	Time	Action	Message
1	19:31:41	SELECT empno, ename, dname FROM emp CROSS JOIN dept LIMIT 0, 1000	56 row(s) returned

❑ ANSI 조인 – Natural Join

- Natural Join
  –Natural Join을 사용할 때의 전제조건은 양쪽 Table에 반드시 1개의 공통 컬럼이 있어야 한다.
  만약 2 개의 공통 컬럼이 있다면, 2개의 공통 컬럼 값이 서로 같은 것만 추출해 온다.

```
SELECT empno, ename, dname
FROM emp NATURAL JOIN dept;
```

```
SELECT empno, ename, dname, deptno
FROM emp e NATURAL JOIN dept d;
```

```
SELECT e.empno, e.ename, d.dname, e.deptno
FROM emp e NATURAL JOIN dept d;
```

- USING(컬럼)

  – 명시적으로 조인으로 사용될 컬럼을 지정하는 방식이다.
  여기서 컬럼명은 조회할 자료가 있는 테이블에 공통된 컬럼명이다.
  –USING 절에서는 컬럼명에 table 별칭을 사용하지 못한다.

```
SELECT empno, ename, dname
FROM emp JOIN dept USING(deptno);
```

```
SELECT e.empno, e.ename, d.dname, e.deptno
FROM emp e JOIN dept d USING(deptno);
```

```
# 에러
SELECT e.empno, e.ename, d.dname, e.deptno
FROM emp e JOIN dept d USING(e.deptno);
```

❑ ANSI 조인 – JOIN~ON

- JOIN~ON 이용한 조인
  - 임의의 조건으로 조인 시 사용한다.
    (서브쿼리 , AND/OR 연산자 , EXIST , IN 연산자)
  - 검색 조건 목적인 경우는 WHERE절을 사용한다.

```
SELECT empno, ename, dname, loc , e.deptno
FROM emp e JOIN dept d ON e.deptno = d.deptno;
```

```
SELECT empno, ename, dname, loc , sal,  e.deptno
FROM emp e JOIN dept d ON e.deptno = d.deptno
WHERE sal > 2500;
```

```
SELECT empno, ename, dname,  sal, grade
FROM emp e JOIN dept d ON e.deptno = d.deptno
           JOIN salgrade s ON e.sal BETWEEN s.LOSAL AND s.HISAL;
```

```
# self 조인
SELECT e.ename 사원, m.ename 관리자
FROM emp e JOIN emp m ON e.mgr = m.empno;
```

❑ ANSI 조인 – Outer Join

- LEFT OUTER JOIN ~ ON | USING

  – 좌측에 기술한 테이블의 모든 행들이 우측에 기술한 테이블내 행들과 일치 여부에 상관없이 모두 출력된다.

- RIGHT OUTER JOIN ~ ON | USING

  – 우측에 기술한 테이블의 모든 행들이 좌측에 기술한 테이블내 행들과 일치 여부에 상관없이 모두 출력된다.

누락된 40 부서번호 포함하여 출력

```
SELECT empno, ename, dname, d.deptno
FROM emp e RIGHT OUTER JOIN dept d ON e.deptno = d.deptno;
```

누락된 신입사원 포함하여 출력

```
# 신입사원 추가
INSERT INTO emp VALUES ( 9000, 'TEST','SALESMAN', 7499,
'90/01/01', 600, NULL, NULL );
```

```
SELECT empno, ename, dname, d.deptno
FROM emp e LEFT OUTER JOIN dept d ON e.deptno = d.deptno;
```

1. 부서 테이블과 사원테이블에서 사번, 사원명, 부서코드, 부서명을 검색 하시오.
(사원명 오름차순 정렬할 것)

Result Grid | Filter Rows:

EMPNO	ENAME	DEPTNO	DNAME
7876	ADAMS	20	RESEARCH
7499	ALLEN	30	SALES
7698	BLAKE	30	SALES
7782	CLARK	10	ACCOUNTING
7902	FORD	20	RESEARCH
7900	JAMES	30	SALES
7566	JONES	20	RESEARCH
7839	KING	10	ACCOUNTING
7654	MARTIN	30	SALES
7934	MILLER	10	ACCOUNTING
7788	SCOTT	20	RESEARCH
7369	SMITH	20	RESEARCH
7844	TURNER	30	SALES
7521	WARD	30	SALES

2. 부서 테이블과 사원테이블에서 사번, 사원명 , 급여 , 부서명을 검색 하시오. 단, 급여가 2000 이상인 사원에 대하여 급여기준으로 내림차순 정렬할 것

Result Grid | Filter Rows:

EMPNO	ENAME	SAL	DEPTNO
7839	KING	5000	10
7788	SCOTT	3000	20
7902	FORD	3000	20
7566	JONES	2975	20
7698	BLAKE	2850	30
7782	CLARK	2450	10

3. 부서 테이블과 사원 테이블에서 사번, 사원명, 업무, 급여 , 부서명을 검색 하시오. 단, 업무가 Manager이며 급여가 2500 이상인 사원에 대하여 사번을 기준으로 오름차순 정렬할 것.

Result Grid | Filter Rows:

EMPNO	ENAME	JOB	SAL	DEPTNO
7566	JONES	MANAGER	2975	20
7698	BLAKE	MANAGER	2850	30

4. 사원 테이블과 급여 등급 테이블에서 사번, 사원명, 급여, 등급을 검색 하시오.
단, 등급은 급여가 하한값과 상한값 범위에 포함되고 등급이 4이며 급여를 기준으로 내림차순 정렬할 것.

Result Grid | Filter Rows:

EMPNO	ENAME	SAL	GRADE
7788	SCOTT	3000	4
7902	FORD	3000	4
7566	JONES	2975	4
7698	BLAKE	2850	4
7782	CLARK	2450	4

5. 부서 테이블, 사원 테이블, 급여등급 테이블에서 사번, 사원명, 부서명, 급여 , 등급을 검색 하시오.
단, 등급은 급여가 하한값과 상한값 범위에 포함되며 등급을 기준으로 내림차순 정렬할 것.

Result Grid | Filter Rows:

EMPNO	ENAME	DEPTNO	SAL	GRADE
7839	KING	10	5000	5
7566	JONES	20	2975	4
7698	BLAKE	30	2850	4
7782	CLARK	10	2450	4
7788	SCOTT	20	3000	4
7902	FORD	20	3000	4
7499	ALLEN	30	1600	3
7844	TURNER	30	1500	3
7521	WARD	30	1250	2
7654	MARTIN	30	1250	2
7934	MILLER	10	1300	2
7369	SMITH	20	800	1
7876	ADAMS	20	1100	1
7900	JAMES	30	950	1

6. 사원 테이블에서 사원명과 해당 사원의 관리자명을 검색 하시오

Result Grid

사원	관리자
SMITH	FORD
ALLEN	BLAKE
WARD	BLAKE
JONES	KING
MARTIN	BLAKE
BLAKE	KING
CLARK	KING
SCOTT	JONES
TURNER	BLAKE
ADAMS	SCOTT
JAMES	BLAKE
FORD	JONES
MILLER	CLARK
TEST	ALLEN

7. 사원 테이블에서 사원명, 해당 사원의 관리자명, 해당 사원의 관리자의 관리자명을 검색 하시오

Result Grid | Filter Rows:

사원	관리자1	관리자2
SMITH	FORD	JONES
ALLEN	BLAKE	KING
WARD	BLAKE	KING
MARTIN	BLAKE	KING
SCOTT	JONES	KING
TURNER	BLAKE	KING
ADAMS	SCOTT	JONES
JAMES	BLAKE	KING
FORD	JONES	KING
MILLER	CLARK	KING
TEST	ALLEN	BLAKE

8. 7번 결과에서 상위 관리자가 없는 모든 사원의 이름도 사원명에 출력되도록 수정 하시오.

Result Grid | Filter Rows:

사원	관리자1	관리자2
SMITH	FORD	JONES
ALLEN	BLAKE	KING
WARD	BLAKE	KING
JONES	KING	NULL
MARTIN	BLAKE	KING
BLAKE	KING	NULL
CLARK	KING	NULL
SCOTT	JONES	KING
KING	NULL	NULL
TURNER	BLAKE	KING
ADAMS	SCOTT	JONES
JAMES	BLAKE	KING
FORD	JONES	KING
MILLER	CLARK	KING
TEST	ALLEN	BLAKE

9. 20번 부서의 이름과 그 부서에 근무하는 사원의 이름을 출력 하시오.

Result Grid | Filter Rows:

ename	dname	deptno
SMITH	RESEARCH	20
JONES	RESEARCH	20
SCOTT	RESEARCH	20
ADAMS	RESEARCH	20
FORD	RESEARCH	20

10. 커미션을 받는 사원의 이름, 커미션, 부서이름을 출력 하시오

Result Grid | Filter Rows:

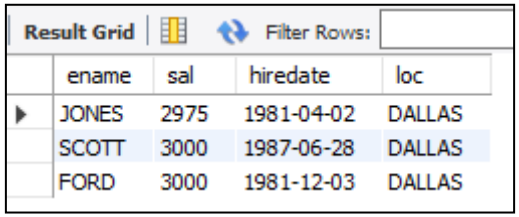
ename	comm	dname
ALLEN	300	SALES
WARD	500	SALES
MARTIN	1400	SALES

11. 이름에 'A' 가 들어가는 사원들의 이름과 부서명을 출력 하시오.

Result Grid | Filter Row

ename	dname
ALLEN	SALES
WARD	SALES
MARTIN	SALES
BLAKE	SALES
CLARK	ACCOUNTING
ADAMS	RESEARCH
JAMES	SALES

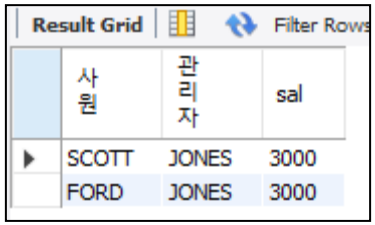
12. DALLAS에 근무하는 사원 중 급여 1500 이상인 사원의 이름, 급여, 입사일, 위치를 출력 하시오.



Result Grid | Filter Rows:

ename	sal	hiredate	loc
JONES	2975	1981-04-02	DALLAS
SCOTT	3000	1987-06-28	DALLAS
FORD	3000	1981-12-03	DALLAS

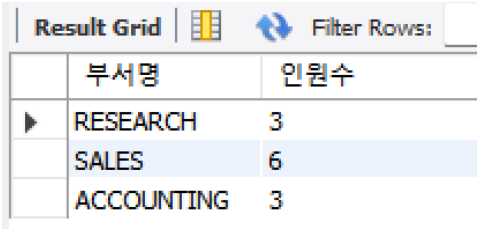
13. 자신의 관리자 보다 연봉(sal)을 많이 받는 사원의 이름, 관리자 이름과 연봉을 출력 하시오.



Result Grid | Filter Rows

사원	관리자	sal
SCOTT	JONES	3000
FORD	JONES	3000

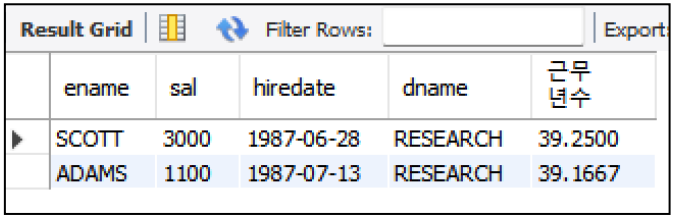
14. 각 부서별로 1982년 이전에 입사한 직원들의 부서명, 인원수를 출력 하시오.



Result Grid | Filter Rows:

부서명	인원수
RESEARCH	3
SALES	6
ACCOUNTING	3

15. 직원 중 현재시간(2026년) 기준으로 근무 년수가 40년보다 적은 사람의 이름, 급여 , 입사일 , 부서명을 출력 하시오.


Result Grid | Filter Rows: | Export:

ename	sal	hiredate	dname	근무 년수
SCOTT	3000	1987-06-28	RESEARCH	39.2500
ADAMS	1100	1987-07-13	RESEARCH	39.1667

chapter 07.

서브쿼리

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/subqueries.html

- 서브쿼리 정의

  - SELECT 문 (Mainquery) 에 포함되어 있는 별도 SELECT 문 (Subquery)이다.
  - 여러 번의 SELECT문을 수행해야 얻을 수 있는 결과를 하나의 중첩된 SELECT 문으로 쉽게 얻을 수 있도록 해준다.

샘플) emp 테이블에서 SCOTT 사원보다 많은 급여를 받는 사원의 이름을 검색 하시오.

```
SQL> SELECT SAL
  2  FROM EMP
  3  WHERE ENAME = 'SCOTT';

       SAL
----------
      3000

SQL> SELECT ENAME
  2  FROM EMP
  3  WHERE SAL > 3000;

ENAME
--------------------
KING
```



```
SQL> SELECT ENAME
  2  FROM EMP
  3  WHERE SAL > ( SELECT SAL
  4                FROM EMP
  5                WHERE ENAME = 'SCOTT' );

ENAME
--------------------
KING
```

- 서브쿼리 기본문법

```
SELECT column_list
FROM table
WHERE   expr   operator  ( SELECT   column_list
                           FROM table );
```

- 바깥 쪽 쿼리를 Main query , 안쪽 쿼리를 Subquery 라고 한다.
- 서브쿼리가 먼저 실행되고, 그 결과가 메인 쿼리에 전달되어 실행된다.
- 서브쿼리는 SELECT,FROM,WHERE,HAVING,ORDER BY,UPDATE,INSERT INTO절에도 사용될 수 있다.
- 연산자는 단일 행 연산자 (> , = , >= , < , <= , !=)와 복수 행 연산자 (IN , ANY ,ALL,EXISTS)를 사용 할 수 있다.
- SQL 문장의 서브쿼리가 단일 행 서브쿼리인 경우는 단일 행 연산자를 사용하고 복수행 서브쿼리인 경우에는 복수 행 연산자를 사용해야 한다.
- 서브쿼리에는 반드시 괄호를 사용한다.
- 서브쿼리에는 ORDER BY 절을 사용할 수 없다.

- 단일행 서브쿼리

  – 서브쿼리가 한 개의 행을 리턴.
  – 반드시 단일행 연산자를 사용해야 한다. (= , > , < , >= , <= , !=)

샘플) EMP 테이블에서 SCOTT의 급여보다 많은 사원의 사원번호, 이름, 담당업무, 급여 출력 ?

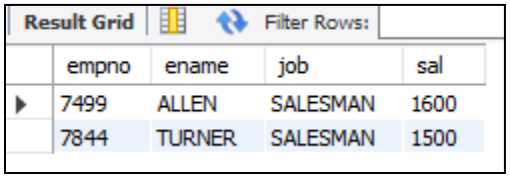
```
SELECT empno, ename, job, sal
FROM emp
WHERE sal > ( SELECT sal
              FROM emp
              WHERE ename = 'scott');
```

Result Grid | Filter Rows:

empno	ename	job	sal
7839	KING	PRESIDENT	5000

샘플) EMP 테이블에서 사원번호가 7521의 업무와 같고 급여가 7934보다 많은 사원의 사원번호, 이름, 담당업무,입사일자, 급여 출력 ?

```
SELECT empno, ename, job, sal
FROM emp
WHERE job = ( SELECT job
              FROM emp
              WHERE empno = 7521 )
AND
      sal > ( SELECT sal
              FROM emp
              WHERE empno = 7934 );
```


Result Grid | Filter Rows:

empno	ename	job	sal
7499	ALLEN	SALESMAN	1600
7844	TURNER	SALESMAN	1500

샘플) EMP 테이블에서 급여의 평균보다 적은 사원의 사원번호, 이름, 담당업무, 급여, 부서번호를 출력 ?

```
SELECT empno, ename, job, sal, deptno
FROM emp
WHERE sal < (SELECT AVG(sal)
             FROM emp );
```

Result Grid | Filter Rows:

empno	ename	job	sal	deptno
7369	SMITH	CLERK	800	20
7499	ALLEN	SALESMAN	1600	30
7521	WARD	SALESMAN	1250	30
7654	MARTIN	SALESMAN	1250	30
7844	TURNER	SALESMAN	1500	30
7876	ADAMS	CLERK	1100	20
7900	JAMES	CLERK	950	30
7934	MILLER	CLERK	1300	10
9000	TEST	SALESMAN	600	NULL

샘플) EMP 테이블에서 20번 부서의 최소 급여보다 많은 모든 부서를 출력 ?

```
SELECT deptno, MIN(sal)
FROM emp
GROUP BY deptno
HAVING MIN(sal) > ( SELECT MIN(sal)
                    FROM emp
                    WHERE deptno = 20 );
```

Result Grid | Filter

deptno	MIN(sal)
10	1300
30	950

❑ 서브쿼리 – 복수행 서브쿼리

- 복수행 서브쿼리

  – 서브쿼리가 여러 개의 행을 리턴.
  – 반드시 복수행 연산자를 사용해야 한다. (IN , ANY , ALL , EXISTS)

- IN 연산자
  – WHERE 절에서 사용하는 일반 비교연산자와 동일하다.
  – 메인쿼리의 비교 조건이 서브쿼리의 결과 중에서 하나라도 일치하면 검색가능.

샘플) EMP 테이블에서 업무별로 최소 급여를 받는 사원의 사원번호, 이름, 업무, 입사일자, 급여, 부서번호를 출력 ?

```
SELECT empno, ename, job, hiredate, sal, deptno
FROM emp
WHERE sal IN ( SELECT MIN(sal)
               FROM emp
               GROUP BY job );
```

Result Grid | Filter Rows: | Export:

empno	ename	job	hiredate	sal	deptno
7369	SMITH	CLERK	1980-12-17	800	20
7782	CLARK	MANAGER	1981-06-09	2450	10
7788	SCOTT	ANALYST	1987-06-28	3000	20
7839	KING	PRESIDENT	1981-11-17	5000	10
7902	FORD	ANALYST	1981-12-03	3000	20
9000	TEST	SALESMAN	1990-01-01	600	NULL

▪ ALL 연산자

–복수행 서브쿼리 결과가 메인 쿼리의 WHERE 절에서 부등호 조건으로 비교할 때 사용된다. (원래 부등호 조건은 단일행에서 사용됨)
–서브쿼리에서 반환되는 행들 전체에 대해 모두 조건을 만족해야 된다.

샘플) 사원 테이블에서 업무가 MANAGER인 사원의 최소급여보다 적은 급여를 받는 사원들의 이름 검색?

```
SQL> SELECT SAL
  2  FROM EMP
  3  WHERE JOB = 'MANAGER';

       SAL
----------
      2975
      2850
      2450
```

최소값보다 작은

```
SQL> SELECT EMPNO, ENAME, SAL
  2  FROM EMP
  3  WHERE SAL < 2450;

     EMPNO ENAME                          SAL
---------- -------------------- ----------
      7369 SMITH                         800
      7499 ALLEN                        1600
      7521 WARD                         1250
      7654 MARTIN                       1250
      7844 TURNER                       1500
      7876 ADAMS                        1100
      7900 JAMES                         950
      7934 MILLER                       1300

8 개의 행이 선택되었습니다.
```

```
SQL> SELECT EMPNO, ENAME, SAL
  2  FROM EMP
  3  WHERE SAL < ALL (SELECT SAL
  4                   FROM EMP
  5                   WHERE JOB = 'MANAGER' );

     EMPNO ENAME                          SAL
---------- -------------------- ----------
      7369 SMITH                         800
      7499 ALLEN                        1600
      7521 WARD                         1250
      7654 MARTIN                       1250
      7844 TURNER                       1500
      7876 ADAMS                        1100
      7900 JAMES                         950
      7934 MILLER                       1300

8 개의 행이 선택되었습니다.
```

▪ ANY 연산자

– 복수행 서브쿼리 결과가 메인 쿼리의 WHERE 절에서 부등호 조건으로 비교할 때 사용된다.
– 서브쿼리에서 반환되는 행들 중에서 어느 하나만 만족해도 된다.

샘플)사원 테이블에서 업무가 MANAGER인 사원의 최소급여보다 많은 급여를 받는 사원들의 이름 검색?

최소값보다 큰

```
SQL> SELECT SAL
  2  FROM EMP
  3  WHERE JOB = 'MANAGER';

       SAL
----------
      2975
      2850
      2450
```

```
SQL> SELECT EMPNO, ENAME , SAL
  2  FROM EMP
  3  WHERE SAL > 2450;

     EMPNO ENAME                          SAL
---------- ------------------- ----------
      7566 JONES                       2975
      7698 BLAKE                       2850
      7788 SCOTT                       3000
      7839 KING                        5000
      7902 FORD                        3000
```

```
SQL> SELECT EMPNO, ENAME , SAL
  2  FROM EMP
  3  WHERE SAL > ANY ( SELECT SAL
  4                    FROM EMP
  5                    WHERE JOB = 'MANAGER');

     EMPNO ENAME                          SAL
---------- ------------------- ----------
      7839 KING                        5000
      7902 FORD                        3000
      7788 SCOTT                       3000
      7566 JONES                       2975
      7698 BLAKE                       2850
```

- ALL 과 ANY 이해

다중행 연산자	설명	동일 기능
> all	최대값 보다 큰 값 반환	> (select max(exp) …)
< all	최소값 보다 작은 값 반환	< (select min(exp) …)
> any	최소값 보다 큰 값 반환	> (select min(exp) …)
< any	최대값 보다 작은 값 반환	< (select max(exp) …)

▪ EXISTS 연산자

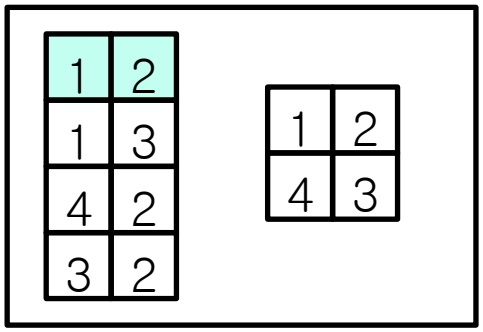
서브쿼리에서 검색된 결과가 하나라도 존재하는지 여부를 확인할 때 사용된다.
만일 서브쿼리에서 검색된 결과가 하나도 없으면 메인 쿼리의 조건절이
거짓이 되어 메인 쿼리가 실행되지 않고 결과가 하나라도 있으면 참이 되어 메인 쿼리가 실행된다.

샘플) 만일 사원중에서 comm을 받는 사원이 한 명이라도 있으면 모든 사원출력?

```
SELECT empno, ename, job, hiredate, sal, deptno
FROM emp
WHERE EXISTS ( SELECT empno
               FROM emp
               WHERE comm IS NOT NULL );
```

▪ 다중 컬럼 서브쿼리

- 서브쿼리에서 여러개의 컬럼값을 검색하여 메인쿼리의 조건절과 비교하는 서브쿼리이다. 메인쿼리의 조건절에서도 서브쿼리의 컬럼수만큼 지정해야 된다.
- 컬럼을 쌍으로 묶어서 동시에 비교한다.



샘플) 부서별로 가장 많은 sal을 받는 사원 정보 출력 ?

```
SELECT deptno, empno, ename,  sal
FROM emp
WHERE (deptno, sal ) IN ( SELECT deptno, MAX(sal)
                          FROM emp
                          GROUP BY deptno);
```

Result Grid | Filter Rows:

deptno	empno	ename	sal
30	7698	BLAKE	2850
20	7788	SCOTT	3000
10	7839	KING	5000
20	7902	FORD	3000

- 서브쿼리 위치에 따른 종류

종류	위치	용도
스칼라 서브쿼리	SELECT, WHERE	단일 값 반환
인라인 뷰	FROM	조회 결과를 임시 테이블처럼 사용
중첩 서브쿼리	WHERE, HAVING	조건 비교 및 필터링

샘플> 각 직원의 정보와 함께 전체 평균 급여 출력?

```
SELECT ename, sal,
       (SELECT ROUND(AVG(sal)) FROM emp) AS avg_salary
FROM emp;
```

Result Grid | Filter Rows:

ename	sal	avg_salary
SMITH	800	2073
ALLEN	1600	2073
WARD	1250	2073
JONES	2975	2073
MARTIN	1250	2073
BLAKE	2850	2073

from절 뒤에 테이블 명이 나와야 되지만, 서브 쿼리가 하나의 가상 테이블을 반환하는 형태로 사용되는 경우를 의미한다.

```
SELECT column_list
FROM (Subquery) 별칭
[WHERE  condition];
```

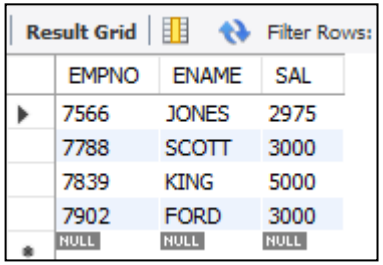
샘플> emp 와 dept 테이블에서 부서별 sal 총합과 평균 및 갯수 출력?

```
SELECT e.deptno, total_sum, total_avg, cnt
FROM ( SELECT deptno, SUM(sal) total_sum, ROUND(AVG(sal)) total_avg,
              COUNT(*) cnt
       FROM emp
       GROUP BY deptno ) e JOIN dept d ON e.deptno = d.deptno;
```

Result Grid | Filter Rows:

deptno	total_sum	total_avg	cnt
10	8750	2917	3
20	10875	2175	5
30	9400	1567	6

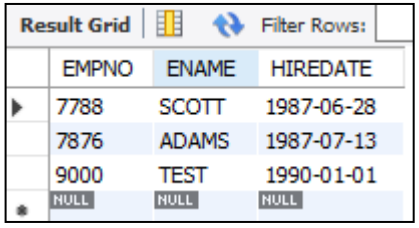
1. 사원 테이블에서 BLAKE 보다 급여가 많은 사원들의 사번, 이름 , 급여를 검색 하시오.



Result Grid | Filter Rows:

EMPNO	ENAME	SAL
7566	JONES	2975
7788	SCOTT	3000
7839	KING	5000
7902	FORD	3000
NULL	NULL	NULL

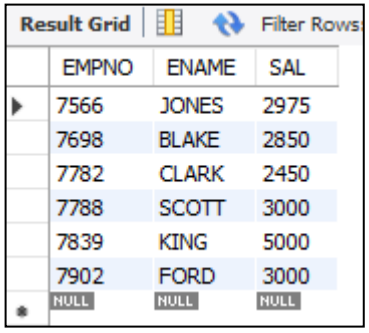
2. 사원 테이블에서 MILLER 보다 늦게 입사한 사원의 사번, 이름, 입사일을 검색 하시오.



Result Grid | Filter Rows:

EMPNO	ENAME	HIREDATE
7788	SCOTT	1987-06-28
7876	ADAMS	1987-07-13
9000	TEST	1990-01-01
NULL	NULL	NULL

3. 사원 테이블에서 사원 전체 평균 급여보다 급여가 많은 사원들의 사번, 이름 , 급여를 검색 하시오



Result Grid | Filter Rows:

EMPNO	ENAME	SAL
7566	JONES	2975
7698	BLAKE	2850
7782	CLARK	2450
7788	SCOTT	3000
7839	KING	5000
7902	FORD	3000
NULL	NULL	NULL

4. 사원 테이블에서 부서별 최대 급여를 받는 사원들의 사번, 이름, 부서코드, 급여를 검색 하시오.

Result Grid | Filter Rows:

EMPNO	ENAME	DEPTNO	SAL
7698	BLAKE	30	2850
7788	SCOTT	20	3000
7839	KING	10	5000
7902	FORD	20	3000
9000	TEST	NULL	600
NULL	NULL	NULL	NULL

5. Salgrade가 2등급인 사원들의 평균 급여보다 적게 받는 사원 정보를 검색 하시오.

Result Grid | Filter Rows: | Edit: | Export/Imp

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800	NULL	20
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7654	MARTIN	SALESMAN	7698	1981-09-28	1250	1400	30
7876	ADAMS	CLERK	7788	1987-07-13	1100	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	950	NULL	30
9000	TEST	SALESMAN	7499	1990-01-01	600	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

chapter 08.

DML

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/sql-data-manipulation-statements.html

- DML 용도 및 종류

13.2 Data Manipulation Statements

13.2.1 CALL Statement
13.2.2 DELETE Statement
13.2.3 DO Statement
13.2.4 EXCEPT Clause
13.2.5 HANDLER Statement
13.2.6 IMPORT TABLE Statement
13.2.7 INSERT Statement
13.2.8 INTERSECT Clause
13.2.9 LOAD DATA Statement
13.2.10 LOAD XML Statement
13.2.11 Parenthesized Query Expressions
13.2.12 REPLACE Statement
13.2.13 SELECT Statement
13.2.14 Set Operations with UNION, INTERSECT, and EXCEPT
13.2.15 Subqueries
13.2.16 TABLE Statement
13.2.17 UPDATE Statement
13.2.18 UNION Clause
13.2.19 VALUES Statement
13.2.20 WITH (Common Table Expressions)

Previous Home Up Next
MySQL Documentation Library

```
# autocommit 설정값 확인
show variables like 'autocommit%';

#autocommit 설정 또는 해제
#SET AUTOCOMMIT = TRUE;     -- 설정
#SET AUTOCOMMIT = FALSE;    -- 해제
```

❑ INSERT – 단일행 입력

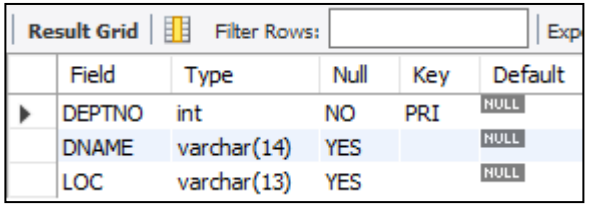
- INSERT 문

– 테이블에 데이터를 입력하기 위한 데이터 조작어 이다.
– 한번에 하나의 행을 테이블에 입력하는 방법과 value_list 및 서브쿼리를 이용하여 한번에 여러 행을 동시에 입력하는 방법이 있다.

```
INSERT INTO table [(column [,column ..])]
VALUES ( value [,value …] );
```

– INTO 절에 명시한 컬럼에 VALUES 절에서 지정한 컬럼값을 입력한다.
(일대일 대응, 일부 컬럼만 저장가능)
– INTO 절에 컬럼명을 지정하지 않으면 테이블 생성시 정의한 컬럼순서와 동일한 순서로 입력된다.
– 입력되는 데이터의 타입은 컬럼의 데이터 타입과 같아야 되며 입력되는 데이터의 크기는 컬럼의 크기보다 작아야 된다.

```
DESC dept;
```



Field	Type	Null	Key	Default
DEPTNO	int	NO	PRI	NULL
DNAME	varchar(14)	YES		NULL
LOC	varchar(13)	YES		NULL

varchar(14): 한글(영문자) 14개 글자까지 저장가능

컬럼명 지정한 경우

```
INSERT INTO dept (deptno, dname, loc )
VALUES (90, '인사과','서울');
```

```
INSERT INTO dept (deptno, dname ) # loc 컬럼에 null 저장
VALUES (91, '인사과');
```

```
INSERT INTO dept (loc, dname, deptno )
VALUES ('서울', '인사과', 80);
```

컬럼명 지정하지 않은 경우

```
INSERT INTO dept
VALUES (70, '인사과','서울');
```

❑ INSERT – 다중행 입력

▪ 다중 행 입력

– 서브쿼리 및 value_lilst 절을 이용하여 하나의 INSERT 명령문에 여러 행을 동시에 저장이 가능하다.

```
# CTAS 이용한 테이블 생성
CREATE TABLE copy_emp
AS
SELECT empno, ename, sal FROM emp
WHERE 1=2;
```

```
# 서브쿼리 이용한 다중 데이터 저장 1
INSERT INTO copy_emp (empno, ename, sal)
SELECT empno, ename, sal
FROM emp;
```

```
# value_list 이용한 다중 데이터 저장 2
INSERT INTO copy_emp (empno, ename, sal)
VALUES (71, '홍길동',800), (72, '이순신',900), (73, '유관순',950);
```

- UPDATE 용도
  - 테이블에 저장된 컬럼들을 변경하는 문장이다.
  - 한 번에 여러 개의 행들을 변경할 수 있다.

```
UPDATE table
SET  column = value [, column = value, … ]
[WHERE condition]
[ORDER BY exp]
[LIMIT row_count];
```

```
UPDATE dept
SET dname ='경리과', loc='부산'
WHERE deptno = 90;
```

```
UPDATE emp
SET sal = 1000
ORDER BY sal DESC
LIMIT 2;
```

- DELETE 용도
  - 테이블에 저장된 행들을 삭제한다.
  - 한 번에 여러 개의 행들을 삭제할 수 있다.

```
DELETE FROM table
[WHERE condition]
[ORDER BY exp]
[LIMIT row_count];
```

```
DELETE FROM dept
WHERE deptno = 90;
```

```
DELETE FROM copy_emp
WHERE sal = 3000
ORDER BY empno desc
LIMIT  1;
```

- MERGE 기능
  - 대상 테이블에 해당 행이 이미 존재하면 UPDATE 가 실행되고 존재하지 않으면 INSERT가 실행된다.

13.2.7.2 INSERT ... ON DUPLICATE KEY UPDATE Statement

If you specify an `ON DUPLICATE KEY UPDATE` clause and a row to be inserted would cause a duplicate value in a `UNIQUE` index or `PRIMARY KEY`, an `UPDATE` of the old row occurs. For example, if column `a` is declared as `UNIQUE` and contains the value `1`, the following two statements have similar effect:

```
INSERT INTO t1 (a,b,c) VALUES (1,2,3)
  ON DUPLICATE KEY UPDATE c=c+1;

UPDATE t1 SET c=c+1 WHERE a=1;
```

```
INSERT INTO dept (deptno, dname, loc )
VALUES (90, '인사과','서울');

INSERT INTO dept (deptno, dname, loc )
VALUES (90, '인사과','서울') ON DUPLICATE KEY UPDATE loc='제주';
```

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/sql-transactional-statements.html

- Transaction 정의

-트랜잭션은 데이터베이스의 논리적인 단위이다.
-트랜잭션은 밀접히 관련되어 분리될 수 없는 한 개 이상의 데이터베이스 조작을 의미한다.
-하나의 트랜잭션에는 하나 이상의 SQL문장이 포함되며 분할 할 수 없는 최소의 단위이다. 그렇기 때문에 전부 적용하거나 전부 취소된다.
즉, 트랜잭션은 'All or Nothing'이다.
-트랜잭션의 대상이 되는 SQL문은 DML문이다.

- COMMIT

– 모든 데이터 변경사항을 데이터베이스에 영구히 반영시키는 명령어.
– 변경전의 데이터는 모든 잃게 된다.
– 모든 사용자들이 트랜잭션 종료 후의 결과를 확인 할 수 있다.
– 트랜잭션이 진행 중이었던 행들에 대한 잠금이 모두 해소되며, 다른 사용자에 의해서 변경이 가능해진다.
– MySQL은 기본적으로 autocommit 으로 설정되어 있다.
set autocommit = true|false 지정 가능

```
# autocommit 설정값 확인
show variables like 'autocommit%';
#autocommit 설정 또는 해제
SET AUTOCOMMIT = TRUE;     -- 설정
SET AUTOCOMMIT = FALSE;    -- 해제
```

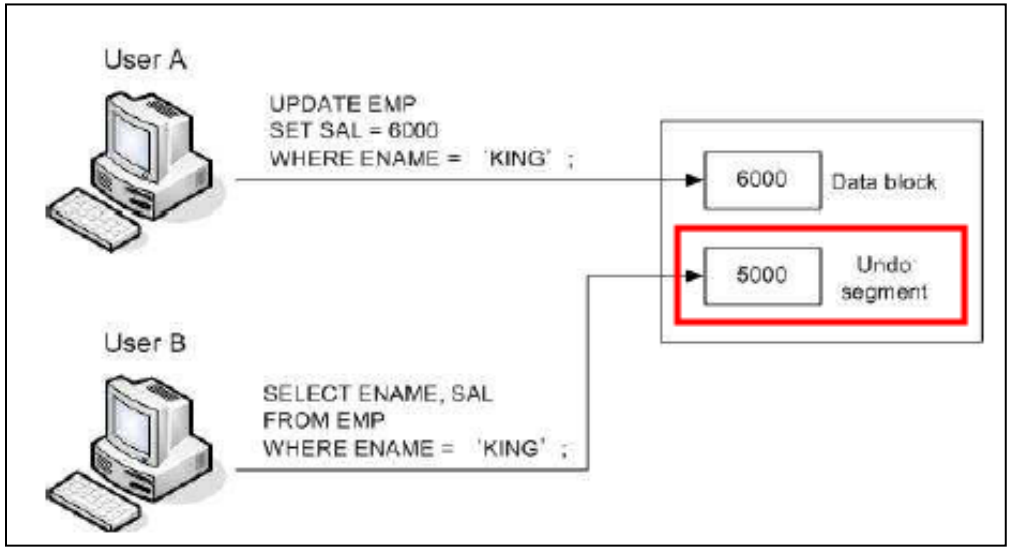
- ROLLBACK

– 모든 데이터 변경사항을 취소하는 명령어.
– 변경전의 데이터로 복원된다.
– 모든 사용자들이 트랜잭션 종료 후의 결과를 확인 할 수 있다.
– 트랜잭션이 진행 중이었던 행들에 대한 잠금이 모두 해소되며 다른 사용자에 의해서 변경이 가능해진다.

▪ 읽기 일관성 (Read Consistency)

- 사용자들에게 가장 최근에 commit 된 데이터를 보여주는 것.
- 데이터를 검색하는 사용자들과 변경하는 사용자들 사이에 데이터의 일관성을 제공한다.
  즉, 다른 사용자들이 변경중인 데이터를 볼 수 없게 한다.(*****)
- 일관적인 데이터베이스 변경방법을 제공함으로써 동일한 데이터를 동시에 변경할 때 발생할 수 있는 혼란을 방지한다.

▪ 읽기 일관성 구현 원리



▪ LOCK 경합

① SQL> SELECT * FROM DEPT;

DEPTNO	DNAME	LOC
90	경리과	부산
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

②
```
SQL> UPDATE DEPT
  2  SET DNAME ='인사과'
  3  WHERE DEPTNO = 90;

1 행이 갱신되었습니다.
```

③ SQL> SELECT * FROM DEPT;

DEPTNO	DNAME	LOC
90	인사과	부산
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

⑥ SQL> ROLLBACK;

④ SQL> SELECT * FROM DEPT;

DEPTNO	DNAME	LOC
90	경리과	부산
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

⑤
```
SQL> UPDATE DEPT
  2  SET LOC = '서울'
  3  WHERE DEPTNO = 90;
```
Wait!!

chapter 09.

DDL

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/sql-data-definition-statements.html

▪ 테이블 생성

```
CREATE TABLE [IF NOT EXISTS] table명
  ( column명 datatype [DEFAULT 값] [AUTO_INCREMENT] [제약조건], … )
```

▪ 데이터 타입

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/data-types.html

정수형 data type

Table 13.1 Required Storage and Range for Integer Types Supported by MySQL

Type	Storage (Bytes)	Minimum Value Signed	Minimum Value Unsigned	Maximum Value Signed	Maximum Value Unsigned
TINYINT	1	-128	0	127	255
SMALLINT	2	-32768	0	32767	65535
MEDIUMINT	3	-8388608	0	8388607	16777215
INT	4	-2147483648	0	2147483647	4294967295
BIGINT	8	-2^{63}	0	$2^{63}-1$	$2^{64}-1$

실수형 data type

float	실수(4Byte) 부동소수점
double	실수(8Byte) 부동소수점
decimal(n1, n2)	고정소수점 n1 : 전체 자리 수, n2 : 소수점 자리 수 ex) 9853.26 -> n1 == 6, n2 == 2

문자형 data type

char(n)	고정형 문자. 저장된 글자 수가 n보다 작으면 공백을 채운다. n은 최대 255
varchar(n)	가변형 문자. 최대 n자의 문자열 저장 가능 n은 최대 65,535
enum(값1, 값2, ...)	정의한 값만 저장할 수 있는 타입

날짜형 data type

Data Type	"Zero" Value
DATE	'0000-00-00'
TIME	'00:00:00'
DATETIME	'0000-00-00 00:00:00'
TIMESTAMP	'0000-00-00 00:00:00'
YEAR	0000

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/sql-data-definition-statements.html

▪ 테이블 생성

```
CREATE TABLE [IF NOT EXISTS] table명
  ( column명 datatype [DEFAULT 값] [AUTO_INCREMENT] [제약조건], … )
```

```
CREATE TABLE IF NOT EXISTS  dept_2
( deptno INT,
  dname VARCHAR(10),
  loc VARCHAR(10) );
```

▪ DEFAULT 옵션

– 해당 테이블에 행을 입력할 때 해당 컬럼에 값을 지정하지 않은 경우 자동으로 기본값이 입력되어 null 값이 저장되는 것을 방지할 수 있다.

```
CREATE TABLE IF NOT EXISTS  dept_3
( deptno INT,
  dname VARCHAR(10),
  loc VARCHAR(10) DEFAULT '서울' );
```

```
CREATE TABLE IF NOT EXISTS board
( num INT PRIMARY KEY AUTO_INCREMENT,
  author VARCHAR(10) NOT NULL,
  title VARCHAR(20) NOT NULL,
  content VARCHAR(100) NOT NULL,
  writeday DATE DEFAULT (CURRENT_DATE),
  readcnt INT DEFAULT 0 );
```

▪ 제약조건 정의 (Constraints)

-부적절한 자료가 입력되는 것을 방지하기 위하여 constraint을 사용한다.
-제약 조건은 종속성이 존재할 경우 테이블 삭제를 방지 한다.
-테이블에서 행이 삽입, 갱신, 삭제될 때마다 테이블에서 규칙을 적용한다.
-제약 조건은 테이블 레벨 및 컬럼 레벨 방법으로 규칙을 적용한다.

▪ 제약조건 종류

제약조건 타입	설명
PRIMARY KEY	테이블의 행(레코드) 식별용. NOT NULL + UNIQUE
UNIQUE	유일한 값 저장. null 허용.
NOT NULL	반드시 값을 저장. null 허용 안함.
CHECK	특정 조건 지정. ex. gender IN ('M','F')
FOREIGN KEY	컬럼과 참조된 테이블의 컬럼 사이의 외래키 관계설정

- 제약조건 확인 방법

```
DESC information_schema.table_constraints;
```

```
SELECT *
FROM information_schema.table_constraints
WHERE TABLE_NAME = 'EMP';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CONSTRAINT_CATALOG	CONSTRAINT_SCHEMA	CONSTRAINT_NAME	TABLE_SCHEMA	TABLE_NAME	CONSTRAINT_TYPE	ENFORCED
def	testdb	PRIMARY	testdb	emp	PRIMARY KEY	YES
def	testdb	emp_ibfk_1	testdb	emp	FOREIGN KEY	YES

- 제약조건 정의 방법

1. 컬럼 LEVEL 지정 방법
   - 한 개의 컬럼에 한 개의 제약조건만 정의 가능하다.
   - FK 제외한 제약조건에 대해서 정의 가능하다. (NOT NULL 제약조건 필수)

```
CREATE TABLE [IF NOT EXISTS] table명
  ( column명 datatype constraint_type,
    column명 datatype constraint_type, … )
```

2. 테이블 LEVEL 지정 방법
   - 테이블의 컬럼 정의와는 별개로 정의한다.
   - 한 개 이상의 컬럼에 한 개의 제약 조건을 정의할 수 있다.
   - NOT NULL 제약조건을 제외한 모든 제약조건 정의 가능하다.

```
CREATE TABLE [IF NOT EXISTS] table명
  ( column명 datatype,
    column명 datatype, …
    COSTRAINT constraint_type, … )
```

❑ 제약조건 (constraint)

▪ PRIMARY KEY

- 하나의 기본 키만이 각 테이블에 대해 존재할 수 있다.
- PRIMARY KEY 제약 조건은 테이블에서 각행을 유일하게 식별하는 열 또는 열의 집합(복합컬럼)이다.(UNIQUE와 NOT NULL 조건을 만족)
- 이 제약 조건은 열 또는 열의 집합의 유일성을 요구하고 NULL값을 포함할 수 없음을 보증 한다.

▪ UNIQUE

- 해당 컬럼에 중복된 값이 저장되지 않도록 제한한다.
- 한 개 이상의 컬럼(복합컬럼)으로 구성 할 수 있다.
- NULL 값 저장 가능하다.

▪ NOT NULL

-해당 컬럼에 NULL 값이 입력되는 것을 방지하는 제약조건이다.
-NOT NULL 제약조건은 컬럼 레벨에서만 지정 가능하다.

▪ CHECK

-해당 컬럼에 반드시 만족해야 될 조건을 지정하는 제약 조건이다. (회사의 업무규칙등)

❑ 제약조건 (constraint)

컬럼 레벨 방식

```
CREATE TABLE student
( no INT  PRIMARY KEY,
  name VARCHAR(10) UNIQUE ,
  address VARCHAR(10) NOT NULL,  # 컬럼레벨만 가능
  age INT CHECK( age IN ( 10,20,30 )));
```

테이블 레벨 방식

```
CREATE TABLE student2
( no INT,
  name VARCHAR(10),
  address VARCHAR(10) NOT NULL,
  age INT ,
  CONSTRAINT  PRIMARY KEY(NO),
  CONSTRAINT  UNIQUE(NAME),
  CONSTRAINT  CHECK( age IN ( 10,20,30 )));
```

- FOREIGN KEY

  - FOREIGN KEY는 child 테이블에서 정의한다.
  - master 테이블의 PRIMARY KEY, UNIQUE KEY로 정의된 열을 참조 할 수 있으며 참조된 열의 값 및 NULL값만 저장할 수 있다.
  - FOREIGN KEY는 열 또는 열의 집합을 지정할 수 있으며 동일 테이블 또는 다른 테이블 간의 관계를 지정할 수 있다.

```
CREATE TABLE MASTER1
( NO INT PRIMARY KEY ,
  NAME VARCHAR(10) NOT NULL );
```

테이블 레벨 방식

```
CREATE TABLE SLAVE1
( num INT PRIMARY KEY,
  NO2 INT,
  CONSTRAINT FOREIGN KEY(NO2) REFERENCES master1(NO));
```

- FOREIGN KEY 추가 옵션
  - 부모 테이블의 행 삭제 시 문제될 수 있는 자식 테이블 행 설정법

ON DELETE CASCADE
- FK 제약조건에 의해 참조되는 테이블(부모 테이블)의 행이 삭제되면 해당 행을 참조하는 테이블(자식 테이블)의 행도 같이 삭제 되도록 한다.

ON DELETE SET NULL
- FK 제약조건에 의해 참조되는 테이블(부모 테이블)의 행이 삭제되면 해당 행을 참조하는 테이블(자식 테이블)의 컬럼을 NULL 로 설정한다.

테이블 레벨 방식

```
CREATE TABLE SLAVE2
( num INT PRIMARY KEY,
  NO INT,
  CONSTRAINT FOREIGN KEY(NO) REFERENCES master1(NO) ON DELETE CASCADE );
```

```
CREATE TABLE SLAVE3
( num INT PRIMARY KEY,
  NO INT,
  CONSTRAINT FOREIGN KEY(NO) REFERENCES master1(NO) ON DELETE SET NULL );
```

▪ 테이블 삭제

- 데이터베이스에서 해당 테이블을 제거하는 것이다.
- 테이블에 저장된 모든 데이터와 관련 INDEX및 제약조건이 삭제된다.
- 만약 foreign key 제약조건이 있으면 테이블 삭제 안됨. CASCADE 지원 안됨.

```
DROP TABLE [IF EXISTS]  table명, …
```

The RESTRICT and CASCADE keywords do nothing. They are permitted to make porting easier from other database systems.

▪ 테이블 잘라내기

-테이블의 모든 행들을 삭제 할 수 있다.
-TRUNCATE은 저장 공간을 해제하고 DELETE 명령은 해제하지 않는다.
-ROLLBACK 정보를 발생시키지 않아서 DELETE 보다 수행속도가 빠르다.
단, DELETE와 달리 ROLLBACk 은 불가능하다.

```
TRUNCATE TABLE table명;
```

❑ 테이블 변경 – 컬럼 추가

- 테이블 변경 (ALTER TABLE 문)

  - 새로운 컬럼 추가
  - 기존 컬럼 변경
  - 컬럼 이름 변경
  - 컬럼 삭제
  - 제약조건 추가 , 삭제
  - 제약조건 활성화 , 비활성화
  - 테이블 읽기 모드(read only)

- 컬럼 추가

```
ALTER TABLE 테이블명
ADD [COLUMN] (column  datatype , … );
```

```
CREATE TABLE scott_t
( num INT,
  NAME VARCHAR(10) );

DESC scott_t;

ALTER TABLE scott_t
ADD ( address VARCHAR(30));
```

- 기존 컬럼 변경

  - 숫자 및 문자 컬럼의 전체 길이를 증가 또는 축소 시킬 수 있다.
    단, 모든 행의 컬럼이 NULL 또는 행이 없는 경우
  - 모든 행의 해당 컬럼 값이 NULL인 경우에만 데이터 타입을 변경 할 수 있다.
  - 디폴트 값을 변경하면 변경 이후부터 입력되는 행에 대해서만 적용된다.

```
ALTER TABLE 테이블명
MODIFY [COLUMN]  column  datatype;
```

```
ALTER TABLE scott_t
MODIFY address VARCHAR(5);

ALTER TABLE scott_t
MODIFY address INT;
```

❑ 테이블 변경 – 컬럼 이름 변경 및 컬럼 삭제

- 컬럼 이름 변경

```
ALTER TABLE 테이블명
RENAME  COLUMN old_column  TO  new_column;
```

```
ALTER TABLE scott_t
RENAME COLUMN address TO addr;
```

- 컬럼 삭제

```
ALTER TABLE 테이블명
DROP  [COLUMN]  column ;
```

```
ALTER TABLE scott_t
DROP address;
```

- 제약조건 추가

-NOT NULL 제약조건은 ALTER TABLE ~ MODIFY 명령을 사용한다.
-나머지 제약조건은 ALTER TABLE ~ ADD 명령을 사용한다.

ALTER TABLE 테이블명
ADD CONSTRAINT 제약조건타입(컬럼);

```
CREATE TABLE scott_t2
( num INT,
  NAME VARCHAR(10),
  age INT,
  address VARCHAR(20) );
```

```
ALTER TABLE scott_t2
ADD CONSTRAINT PRIMARY KEY(num);

ALTER TABLE scott_t2
ADD CONSTRAINT UNIQUE(NAME);

ALTER TABLE scott_t2
ADD CONSTRAINT CHECK(age>20);

ALTER TABLE scott_t2
MODIFY address VARCHAR(20) NOT NULL;
```

```
CREATE TABLE IF NOT EXISTS scott_t2_child
( no INT PRIMARY KEY,
  num INT );
```

```
ALTER TABLE scott_t2_child
ADD CONSTRAINT FOREIGN KEY(num)
                REFERENCES scott_t2(num);
```

Thank you